

Contents

interTempFoam – Publication-Grade Technical Audit	2
1. Architecture overview	2
1.1 PIMPLE coupling order (interTempFoam.C)	2
1.2 The three HW physics pathways	3
2. Coupling / source-term dependency map	3
3. Critical inconsistency – Helmholtz redistribution is partially dead	4
3.1 Recommended resolution	4
4. qn / one-sided gradient reconstruction audit	4
4.1 Dead code path – TSense	4
4.2 The neighbor-search reconstruction (:702–765)	4
4.3 Recommended qn reconstruction	6
4.4 kEff inconsistency	7
5. TEqn audit (TEqn.H)	7
5.1 Current structure	7
5.2 Findings	7
5.3 Proposed clean TEqn	8
6. Solver-level (interTempFoam.C) audit	9
6.1 phaseChangePtr->correct() called only on pimple.firstIter()	9
6.2 phiTotal updated only in outer loop, not in inner	9
6.3 rho, rhoCp recomputed manually after pEqn	9
6.4 pcDictPtr outer-scope hack	9
7. alphaEqn / alphaSuSp audit	9
7.1 Su=0 even though alpha1Gen is computed	9
7.2 phiStefan in flux compression	10
7.3 phiTotal vs phiCN inside MULESCorr	10
7.4 Commented-out divergence subtraction	10
8. pEqn / continuity audit	10
8.1 PCV correctness	10
8.2 phiStefan not in phiHbyA	10
8.3 Missing Rhie-Chow consistency check	11
9. Empirical regularization audit (full inventory)	11
10. Numerical concerns (prioritized)	12
10.1 Critical (publication-blocking)	12
10.2 High impact	12
10.3 Medium impact	12
10.4 Low impact / cleanup	13
11. Recommended implementation roadmap	13
12. Summary of recommended state after audit	13
13. Defence of smoothing – what is and isn’t justifiable	13
13.1 Why a diffuse-interface Stefan solver needs <i>some</i> smoothing	14
13.2 The diffusion-of-field-variable identity	14
13.3 Per-operation defence	15
14. Why the current Deltat-dependent terms exist (and how to eliminate them)	16
15. Clean Stefan solver architecture (proposed)	16
15.1 Design principles	16
15.2 Mathematical structure	16
15.3 Why no Deltat appears anywhere	17
15.4 Asymptotic limit $h \rightarrow 0$	18
15.5 Implementation footprint	18
16. Generalization to arbitrary phase change	19
17. Summary – the defensible minimum	19
18. Directionality analysis of the phiStefan vector	20

18.1	Direction of $\mathbf{nHat}$	20
18.2	Sign of $\mathbf{Ustef}$	20
18.3	Direction of interface motion	20
18.4	Face-orientation convention	21
18.5	Directional weaknesses	21
18.6	Verdict	22
19.	UEqn and pEqn implementation review	22
19.1	UEqn (UEqn.H)	22
19.2	pEqn (pEqn.H)	23
19.3	Joint UEqn-pEqn consistency	24
20.	alphaEqn source structure – Su, Sp, divU vs phiStefan and PCV	24
20.1	The user’s suspected redundancy	24
20.2	Mathematical derivation	25
20.3	Two equivalent implementations	25
20.4	Where redundancy WOULD live, and why it doesn’t	26
20.5	Quantitative consistency check	26
20.6	What IS suspect in alphaSuSp.H / alphaEqn	27
20.7	Verdict	27
21.	Acoeff in TEqn – what it actually does	28
21.1	What you see in the code	28
21.2	The trick being used: implicit source linearization	28
21.3	Three protective max operations	29
21.4	The implicitFactor = 0.5 mystery	29
21.5	Why the soft-Dirichlet view is physically defensible	30
21.6	The Acoeff_floor and Acoeff_hot_floor add-ons	30
21.7	Summary diagram	31
21.8	What to change for publication	31
21.9	Connection back to the rest of the solver	32
22.	Q_cond – is it required for a generalized bidirectional solver?	32
22.1	Why the current split exists	32
22.2	Bidirectional Acoeff – one expression for both signs	32
22.3	Why this is preferable to the current split	33
22.4	Equivalent cleaner form using pos/neg functions	33
22.5	What changes in the cleaned TEqn	34
22.6	Combined with the other Sec.21 cleanups	34
22.7	Verdict	35

interTempFoam – Publication-Grade Technical Audit

Diffuse-interface Stefan VOF solver with Hardt-Wondra (HW) Helmholtz redistribution. Audit covers `interTempFoam.C`, `createFields.H`, `alphaEqn{,SubCycle}.H`, `alphaSuSp.H`, `UEqn.H`, `pEqn.H`, `TEqn.H`, `thermalPhaseChangeModel{.H,.C}`, `HardtWondra{.H,.C}`.

Audit date: 2026-05-27. Audited against publication scrutiny.

1. Architecture overview

1.1 PIMPLE coupling order (interTempFoam.C)

```
while pimple.loop():
    (firstIter) mesh.update();
    (firstIter) phaseChangePtr->correct();           // builds qn, mdotRaw, mdot, Q_pc, Q_pc_thermal, phiS
    phiTotal = phi + phiStefan;                     // shared by alphaEqn
```

```

alphaEqnSubCycle -> alphaEqn (MULES, phiTotal) // VOF transport
mixture.correct(); // rho, mu from new alpha
UEqn (no Stefan body force; phiStefan enters only via alphaEqn and pEqn)
while pimple.correct():
    pEqn (Laplacian == div(phiHbyA) - PCV) // PCV = volume-dilatation source
    rho, rhoCp recomputed
    TEqn (rhoCpPhi=phi*rhoCp_f, NOT phi+phiStefan) // energy

```

1.2 The three HW physics pathways

Pathway	Field used	Where injected	Purpose
Kinematic	mdotRaw_ (then Ustef = phiStefan -> mdotRaw/(Ai+AiFloor_kin))	alphaEqn phiTotal	Pure interface translation
Thermodynamic (latent)	Q_pc_ = mdotRaw_ * h_lv (and Q_pc_thermal_ for TEqn implicit penalty)	TEqn Acoeff (implicit) + Qcorr (explicit)	Latent heat coupling
Dynamic (continuity dilatation)	PCV = Q_pc/h_lv * (1/rho_2-1/rho_1)	pEqn RHS	Mass-conserved volume source

The architecture is **partially Helmholtz-redistributed** – Helmholtz mdot_ exists, is solved, but is *no longer load-bearing*. See Sec.3.

2. Coupling / source-term dependency map

```

T ----+
      +---> TSense=max(T,Tsat) -> gradT (DEAD, see Sec.4.1)
      |
alpha ----> alphaSmooth -> alphaGeom -> gradalpha -> Ai = |gradalpha|
      |                                     -> nHat -> nHatSmooth
      |                                     -> alphaThermal = 1/2(1+tanh 4(alpha-1/2))
      |                                     -> kEff (harmonic, biased)
      |
T,alpha --> cellCells one-sided search -> dTdn_hot (kinematic, floor=kinFloorCells*h)
      |                                     -> dTdn_hot_thermal (thermal, floor=h_ref)

dTdn_hot -> qn = kEff*dTdn_hot * interfaceMask
      mdotRaw = Ai*qn / h_lv
      +---> Helmholtz solve -> mdot_ (used only by Qcorr & diagnostics)
      +---> mdotInterfacial = mdotRaw/(Ai+AiFloor_kin)
      | -> Ustef = mdotInt_f / rho_int -> phiStefan = Ustef*(nHat_f*Sf)*maskF
      +---> Q_pc = mdotRaw*h_lv -> TEqn Acoeff (via Q_pc_thermal), PCV, alpha1Gen
      +---> (Q_pc_thermal feeds TEqn Acoeff implicit penalty only)

mdot_ (Helmholtz)-> Qcorr = -mdot*(cp_1-cp_2)*(T-Tsat) -> TEqn RHS
phiStefan -> alphaEqn.phiTotal
PCV -> pEqn RHS

```

3. Critical inconsistency – Helmholtz redistribution is partially dead

Hardt & Wondra (2008) constructs $\text{mdot} = \text{mdotRaw} - \lambda^2 \text{grad}^2 \text{mdot}$ so that the *volumetric latent source* $Q_{pc} = \text{mdot} * h_{lv}$ is smooth and spreads into bulk cells. Eq. (42) of HW (the Q_{corr} enthalpy correction) is **mathematically derived assuming Q_{pc} uses the redistributed mdot** , because Q_{corr} removes the artificial enthalpy carried by mass deposited outside the physical phase-change band.

Current code (HardtWondra.C:1122):

```
Q_pc_ = mdotRaw_ * h_lv_; // localized
...
Qcorr_ = -mdot_ * (mixture_.cp1() - mixture_.cp2()) * (T_ - T_sat_); // redistributed
```

This is **internally inconsistent**:

- If Q_{pc} is localized (uses $\text{mdotRaw}_$), then by HW’s derivation $Q_{corr} == 0$. The current $Q_{corr}_$ non-trivially modifies the energy budget by $\sim 1\text{--}2\%$ using a mass field ($\text{mdot}_$) that does not appear elsewhere in the energy equation.
- The Helmholtz solve ($\text{fvm}::\text{laplacian}(\lambda^2)$) is now load-bearing only through $Q_{corr}_$. If $\text{useEnthalpyCorrection}=\text{false}$, the entire Helmholtz solve is dead computational cost.

3.1 Recommended resolution

Pick one consistent variant. The cleanest defensible choice is “localized HW” – drop Helmholtz entirely and remove Q_{corr} (set to zero or remove). Helmholtz redistribution was historically introduced to broaden the latent sink for stability; the current code already broadens via the diffuse VoF band itself, the α_{Thermal} tanh sharpening, and the A_{coeff} hot/cold floors in TEqn.H . Removing $\text{mdot}_/Q_{corr}_$ simplifies the model, eliminates the inconsistency, removes one elliptic solve per outer iteration, and aligns with the existing kinematic + thermodynamic + dynamic pathways which all already use $\text{mdotRaw}_$.

Alternative (faithful HW): use $Q_{pc} = \text{mdot} * h_{lv}$ (redistributed), keep Q_{corr} , but then *also* use mdot (not $\text{mdotRaw}_$) in phiStefan and PCV . This restores HW-consistency at the cost of reintroducing the $\sim 25\%$ interface-position underprediction documented in the existing in-source comments (HardtWondra.C:1001–1014).

4. qn / one-sided gradient reconstruction audit

4.1 Dead code path – TSense

HardtWondra.C:485–525 constructs a smoothed sensing field TSense and computes $\text{gradT} = \text{fvc}::\text{grad}(\text{TSense})$. **Neither TSense nor gradT is used downstream.** $\text{qn}_$ is assembled at :786 from $\text{qnSigned} = k_{\text{Eff}} * d_{\text{Tdn_hot}}$ where $d_{\text{Tdn_hot}}$ comes from the manual $\text{cellCells}()$ neighbor search at :702–765. The TSense diffusion loop (one $\text{fvc}::\text{laplacian}$ per iteration) and the $\text{fvc}::\text{grad}(\text{TSense})$ are pure runtime cost without effect.

Action: delete TSense , D_{Tsense} , the TSense smoothing loop, and the $\text{fvc}::\text{grad}(\text{TSense})$ block.

4.2 The neighbor-search reconstruction (:702–765)

Logic: - For every interface cell ($A_i > \text{SMALL}$): - scan face-neighbors via $\text{cellCells}()$, - reject neighbors with $\text{normalProj} = |\text{dVec} * \text{nHat}| < \text{SMALL}$ (tangential), - reject neighbors with $T_{\text{nbr}} - T_{\text{sat}} < 1e-6$ (cold side), - among survivors, **select the closest** (if ($\text{normalProj} < \text{bestDistance}$)), - compute $\text{grad} = (T_{\text{nbr}} - T_{\text{sat}}) / \max(\text{normalProj}, \text{floor} * h_{\text{ref}})$.

4.2.1 Strengths

- Genuinely one-sided: excludes cold-side neighbors, so the cold-phase T_{sat} pin no longer corrupts the gradient. This is the correct response to diffuse-interface “both phases contaminate gradT ” issue.

- Direct Dirichlet anchor at T_{sat} : gradient is $(T_{hot} - T_{sat}) / d_{hot}$, matching the sharp Stefan condition at the interface plane.
- Floor on the denominator prevents divide-by-tiny-distance singularities.

4.2.2 Weaknesses (publication-critical)

Weakness	Impact	Severity
W1. <code>cellCells()</code> returns face-neighbors only (no corner). On hexahedral meshes the search has 6 candidates; on poorly-aligned interfaces the optimal hot neighbor in the <code>nHat</code> direction may not be a face neighbor.	Direction-dependent qn bias, anisotropic Stefan speed	High
W2. Selection criterion is closest hot neighbor , not best-aligned. A near-tangential face-neighbor with small <code>normalProj</code> can beat a well-aligned but farther neighbor.	Spurious selection switching -> noise in qn	High
W3. Single-neighbor stencil: the gradient is a 2-point one-sided difference. Truncation error $O(h)$. No least-squares fit, no multi-cell extrapolation.	First-order accuracy in qn -> first-order accuracy in interface speed	Critical
W4. <code>kinFloorCells_*h_ref</code> floor (default 2.0) artificially weakens the gradient when the chosen neighbor is closer than 2 cells. On well-resolved meshes this systematically <i>under</i> -predicts qn at exactly the cells doing the work.	Documented 25 % interface-position underprediction (per in-source comments)	Critical
W5. Boundary cells: a hot neighbor across a wall patch is excluded (<code>cellCells</code> does not cross patches). If hot phase abuts a wall, no qn is reconstructed there.	Wall-attached interfaces have artificially zero Stefan velocity	High
W6. Uses T_* directly, not a smoothed field. Combined with VOF staircase and the saturation pin from the anti-trench floor, gradient can flip sign cell-to-cell.	Noise -> <code>mdotRaw</code> oscillation -> Helmholtz solve noise -> <code>Qcorr</code> noise	Medium
W7. The “10x floor” comment is stale; default <code>kinFloorCells_ = 2.0</code> is hardcoded but the comments describe the original 10x behavior.	Documentation drift; defensible parameter choice obscured.	Low (but publication-blocking)

Weakness	Impact	Severity
W8. <code>Q_pc_thermal_</code> uses <i>the same</i> neighbor selection but with a different (smaller) floor. Two latent sources differing only in a magic denominator floor have no physical interpretation.	Cannot justify in publication.	Critical

4.3 Recommended qn reconstruction

A defensible diffuse-interface one-sided gradient with $O(h^2)$ -leaning behavior that remains pure FV:

Algorithm (per interface cell c):

1. Build a hot-phase weight $w_n = \alpha_{\text{hot}}[n] * \text{pos}(T[n] - T_{\text{sat}} - \epsilon)$ for each face neighbor n . α_{hot} is α or $1-\alpha$ depending on which side is hot.
2. Solve a **least-squares hot-side gradient** using only weighted hot neighbors:

$$\text{minimize } \sum_n w_n * ((T_n - T_c) - \text{gradT_LS} * (x_n - x_c))^2$$
3. Project: $dT_{\text{dn}} = \text{gradT_LS} * n_{\text{Hat_c}}$, then clip $dT_{\text{dn}} = \max(dT_{\text{dn}}, 0)$ (cold-side bleed-through removed by the weights, but a safety floor remains).
4. Compute qn as $q_n = k_{\text{Eff}} * dT_{\text{dn}}$. The k_{Eff} at the interface cell is replaced by the *hot-side* k (k_{liq} for melting, k_{vap} for evaporation), not the harmonic mean – physically the conductive flux is being carried by the hot phase up to the interface plane.
5. Replace the magic-number floor on `normalProj` with a tiny dimensionful regularizer $\sqrt{h_{\text{ref}}^2 + (n_{\text{Hat}} * \text{grad}\alpha / |\text{grad}\alpha|)^2}$ that goes to zero on aligned interfaces – eliminates W4 entirely.

OpenFOAM-compatible pseudocode (single-pass, drop-in for :702-765):

```

const scalar TsatVal = T_sat_.value();
forAll(mesh_.cells(), cellI)
{
    if (interfaceArea_[cellI] < SMALL) continue;
    const vector& nh = nHat[cellI];
    const labelList& nbr = mesh_.cellCells()[cellI];
    // Hot-side weighted LS gradient
    symmTensor M = Zero; vector b = Zero;
    forAll(nbr, k) {
        label j = nbr[k];
        scalar w = max(T_[j] - TsatVal, 0.0); // hot-only weight
        if (w < SMALL) continue;
        vector d = mesh_.C()[j] - mesh_.C()[cellI];
        M += w * sqr(d); // symmetric outer product
        b += w * (T_[j] - TsatVal) * d; // anchor: T_iface = Tsat
    }
    if (det(M) > SMALL) {
        vector gT = inv(M) & b;
        dTdn_hot[cellI] = max(gT & nh, 0.0);
    }
}

```

Benefits: - Second-order via LS on whatever neighbor set is available, including more than one hot neighbor.
- Single floor ($\det(M) > \text{SMALL}$) – no magic mesh-units constant. - Single gradient field, no thermal/kinematic

split; eliminates the entire `Q_pc_thermal_` apparatus. - Wall-adjacent cells degrade gracefully: weight goes to zero if all hot neighbors are wall-blocked, `qn` -> 0 with no spurious activation.

4.4 kEff inconsistency

HardtWondra.C:436-459 (used for `qn`) uses a *biased* harmonic mean with `harmonicBias_=0.3` over `alphaThermal = 1/2(1+tanh 4(alpha-1/2))`. `kappaf()` (used for TEqn Laplacian, :1224-1262) uses an *unbiased* harmonic mean over raw `alpha1_`. Two different effective conductivities for the same physics in the same solve. **Recommendation:** collapse to a single `kEff` field (raw harmonic over `alphaThermal`) and reuse it for both `qn` and TEqn. If sharper thermal interface is required, sharpen `alphaThermal`, not introduce a second field. Document `alphaThermal` sharpening with a numerical truncation analysis in the paper.

5. TEqn audit (TEqn.H)

5.1 Current structure

$$\text{rhoCp} * dT/dt + \text{grad}(\text{rhoCp} \text{ phi } T) - \text{grad}(k_f \text{ grad}T) + (A_{\text{main}} + A_{\text{floor}} + A_{\text{hot}}) T \\ = (A_{\text{main}} + A_{\text{floor}} + A_{\text{hot}}) * T_{\text{sat}} - Q_{\text{cond}} + Q_{\text{corr}}$$

with three implicit penalties:

Coeff	Form	Active where	Purpose
<code>Acoeff</code>	$1/2 * \max(Q_{\text{pc_thermal}} / (T - T_{\text{sat}} + \epsilon_T), 0)$	interface band	Latent sink (Lee-style hidden inside HW)
<code>Acoeff_floor</code>	$5 * \text{rhoCp} / \text{Deltat} * 4\alpha(1-\alpha) * \text{pos}(T_{\text{sat}} - T)$	Diffuse band, $T < T_{\text{sat}}$	Anti-trench (cold-side floor)
<code>Acoeff_hot_floor</code>	$5 * \text{rhoCp} / \text{Deltat} * 4\alpha(1-\alpha) * \text{pos}(T - T_{\text{sat}}) * \text{coldSideGate}$	Diffuse band, $T > T_{\text{sat}}$, cold-phase side	Anti-superheat (hot-side floor)

5.2 Findings

F1 (Critical, mathematical). `Acoeff = Q_pc_thermal / (T - Tsat + eps_T)` re-derives a Lee model from HW data: in cells where `Acoeff` is active, the implicit penalty drags $T \rightarrow T_{\text{sat}}$ at rate `Acoeff/rhoCp`. Because `Q_pc_thermal` itself depends on $T_{\text{nbr}} - T_{\text{sat}}$ (via `dTdn_hot_thermal`), the construction is self-consistent *only when solved tightly*. PIMPLE outer-loop convergence is not enforced; with a single outer iteration the system is effectively explicit. **Recommendation:** either (i) make `Q_pc_thermal` explicit and treat the latent term purely as RHS source (no `Acoeff`), or (ii) iterate the phase-change update inside the PIMPLE outer loop (call `correct()` every outer iter, not only `firstIter()` – see Sec.6.2).

F2 (Critical, conservation). `Q_cond = min(Q_pc_now, 0)` peels off the condensation branch as explicit, while `Acoeff*Tsats` provides the evaporation branch implicitly. The split is asymmetric: condensation gets no implicit pull-to- T_{sat} . For pure evaporation cases (Sheikh Stefan) `Q_cond` is identically zero and this branch is dead. Document or remove for single-direction studies.

F3 (Critical, defensibility). `Acoeff_floor` and `Acoeff_hot_floor` are **numerical bandaids** for residual inconsistency in `qn` / `mdotRaw`. They use `rhoCp/Deltat` scaling – i.e. pure time-step-rate penalty – which is the fingerprint of an unstable underlying source being suppressed. Both floors should **not appear** in a clean HW solver. Their presence indicates the `qn` reconstruction (Sec.4) is releasing latent heat in cells where the temperature state then violates T_{sat} bounds.

The physically defensible fix: improve qn reconstruction (LS hot-side gradient, Sec.4.3) and let the Acoeff term alone enforce Tsat in the diffuse band. The floors should be **off by default** and only enabled as an emergency diagnostic, with a runtime warning when any cell triggers them.

F4 (Medium, code-cleanup). The dead `// REPLACE: const volScalarField coldSideGate(...)` block at TEqn.H:51-58 should be deleted. The `coldSideGate` constructor uses an awkward immediate `IObject` + post-assignment pattern; collapse to a single ternary `volScalarField` constructor.

F5 (Medium, conservation). `rhoCpPhi = phi * rhoCp_f` (no `phiStefan`). Comment at `alphaEqn.H:274-278` explicitly justifies this: sensible enthalpy should not ride the Stefan velocity. **Correct decision**, but creates a hidden conservation drift: `d(rhoCp)/dt` from `mixture.correct()` after `alphaEqn` includes interface motion driven by `phiTotal=phi+phiStefan`, while `grad*(rhoCpPhi*T) = grad*(phi*rhoCp_f*T)` uses only `phi`. The implicit assumption is that latent heat from interface motion is captured exactly by `Q_pc` and `Qcorr`. Add a consistency check function-object that monitors `intrhoCp*T dV - int(...) flux - intQ_pc + intQ_cond` per timestep.

F6 (Low, performance). `Q_pc_now`, `Q_pc_th_now`, `Acoeff`, `Q_cond`, `indicator_TEqn`, `Acoeff_floor`, `coldSideGate`, `Acoeff_hot_floor` are all constructed as named `volScalarFields` every TEqn call. With `AUTO_WRITE` off this is OK but it's still 6+ field allocations per outer iteration. Move all to internal-field arithmetic with `volScalarField::Internal` to halve memory traffic.

5.3 Proposed clean TEqn

```
const surfaceScalarField kappaf = phaseChangePtr->kappaf();
const dimensionedScalar Tsat = phaseChangePtr->T_sat();

// Single, explicit latent source. No A_floor, no A_hot, no Q_cond split.
// Latent sink enforced via Acoeff alone, derived from HW Q_pc.
const dimensionedScalar epsDT("epsDT", dimTemperature, scalar(1.0));
const volScalarField::Internal Acoeff
(
    max(phaseChangePtr->Q_pc()/max(T - Tsat, epsDT),
        dimensionedScalar(dimensionSet(1,-1,-3,-1,0,0,0), 0))
);

fvScalarMatrix TEqn
(
    rhoCp*fvm::ddt(T)
  + fvm::div(rhoCpPhi, T)
  - fvm::laplacian(kappaf, T)
  + fvm::Sp(Acoeff, T)
  ==
    Acoeff*Tsat
  + phaseChangePtr->Qcorr()    // becomes ==0 once Sec.3 cleanup applied
);
TEqn.relax();
TEqn.solve();
```

Anti-trench and anti-superheat floors should be **case-level diagnostics** (printed when triggered) rather than baked-in stiffness sources. If they are load-bearing for stability in any production case, that case fails to demonstrate the HW formulation itself converges – a publication-level red flag.

6. Solver-level (interTempFoam.C) audit

6.1 phaseChangePtr->correct() called only on pimple.firstIter()

interTempFoam.C:152-156. Implications: - The phase-change source is frozen across pimple correctors. With a single outer iteration (nOuterCorrectors=1) the energy equation uses sources computed from T^n , not the converging $T^{(n+1,k)}$. The implicit $A_{coeff}*(T - T_{sat})$ partially compensates, but does not close the T-feedback into qn. - For research-grade time integration, call `correct()` every outer iter. Add a dictionary flag `correctPhaseChangeEveryOuter` (default false for backward compat) and recommend `true` in publication runs.

6.2 phiTotal updated only in outer loop, not in inner

interTempFoam.C:158. `phiTotal = phi + phiStefan` is computed before the `alphaEqn` solve. After `pEqn` updates `phi`, `phiTotal` is stale until the next outer iteration. For interface advection this is fine (`alphaEqn` already ran), but it makes the `rhoPhi/rhoCpPhi` calculations inside `alphaEqn` use pre-`pEqn` `phi`. This is the standard `interFoam` pattern, but it means the HW Stefan velocity advecting alpha was paired with an unconverged Darcy velocity.

6.3 rho, rhoCp recomputed manually after pEqn

interTempFoam.C:180-186 recomputes `rho` and `rhoCp` after `pEqn` solves. Then `alphaEqnSubCycle.H:44-47` also recomputes them after `alphaEqn`. The post- `pEqn` rebuild is the live one for `TEqn`; the post-alpha rebuild is overwritten. **Cleanup:** remove the post-alpha rebuild from `alphaEqnSubCycle.H`, or remove the post-`pEqn` rebuild in `interTempFoam.C` – keep one. The post-`pEqn` version is the right one for `TEqn` consistency (alpha is finalized).

6.4 pcDictPtr outer-scope hack

`createFields.H:65-94, 251-257`. The `IOdictionary` is kept alive in `createFields.H` outer scope only so `TEqn.H` can read `coldPhaseIsHighAlpha1`. Two clean alternatives: 1. Move `coldPhaseIsHighAlpha1` into `thermalPhaseChangeModel` base class as a virtual `bool isColdPhaseHighAlpha1()` `const` accessor. Eliminates the dictionary leak. 2. Read the flag once in `createFields.H` and pass it as a `const bool` – already done at :252. Then `pcDictPtr` can be a local scope variable.

Option 1 is the correct architectural choice.

7. alphaEqn / alphaSuSp audit

7.1 Su=0 even though alpha1Gen is computed

`alphaSuSp.H:8-32` sets `Su` to identically zero, but `thermalPhaseChangeModel::alpha1Gen()` returns $-Q_{pc}/(\rho_1 h_{lv})$ – a non-zero phase-change alpha-source. The current model relies entirely on `phiStefan` advection for interface motion; `alpha1Gen` is computed but *never injected into MULES*. This means:

- `alpha1Gen()` is **dead code** in the current solver. The comment in `alphaSuSp.H:1` (“`Su = alpha1Gen`: positive for condensation...”) is false.
- The MULES `Su/Sp` boundedness machinery is bypassed for the volumetric phase-change source. Boundedness is enforced only through `phiStefan` geometry + the post-MULES clip `alpha1.max(0)/min(1)`.

This is a defensible design choice (kinematic-only interface motion is more mesh-convergent than dual-injection `Su+phiStefan`), but it must be: - documented in the paper’s “model description” section, - enforced by either (i) deleting `alpha1Gen()` from the base class or (ii) renaming to `alpha1GenUnused()` to prevent future re-injection.

7.2 phiStefan in flux compression

alphaEqn.H:91-109. The compression flux $\phi_{iCN} = \phi + \phi_{iStefan}$ (or its CN blend). The compressive face flux $\phi_{iR} = \phi_{iCN} n_{Hatf}$ does **not** include $\phi_{iStefan}$. So: - Bulk transport: $\phi_{iTotal} = \phi + \phi_{iStefan}$ OK - MULES upwind: $\phi_{iCN} = \phi_{iTotal}$ OK - Compression: ϕ_{iR} on ϕ -derived ϕ_{iCN} , but applied to α_1 (not α_2) OK standard interFoam

Consistent.

7.3 phiTotal vs phiCN inside MULESCorr

alphaEqn.H:111-171 (MULESCorr branch) uses ϕ_{iCN} (which includes $\phi_{iStefan}$) for the upwind matrix. Later, :180-192 (corrector loop) uses ϕ_{iTotal} for `fvc::flux(phiTotal, alpha1, alphaScheme)`. **Both paths include phiStefan** but the variable names differ (ϕ_{iCN} vs ϕ_{iTotal}). Cleanup: unify naming – pass ϕ_{iTotal} everywhere, or always use ϕ_{iCN} .

7.4 Commented-out divergence subtraction

alphaEqn.H:128-129:

```
// - fvm::Sp(fvc::ddt(dimensionedScalar("1", dimless, 1), mesh)
//           + fvc::div(phiCN), alpha1)
```

This is the standard interFoam non-conservative correction for compressible flux divergence. With $\text{divU} = \text{PCV}$ injected via the `==` source (:131), this term is redundant *if* divU correctly cancels `fvc::div(phiCN) - fvc::div(phi)`. **Verify by derivation:**

$\text{grad}*\phi_{iTotal} = \text{grad}*\phi + \text{grad}*\phi_{iStefan}$. From mass continuity with phase change, $\text{grad}*U = (1/\rho_v - 1/\rho_l) * \dot{m} = \text{PCV}$. So $\text{grad}*\phi = \text{PCV} * V_{cell}$ (in discrete form, when ϕ is volume flux). With $\dot{m} = \dot{m}_{Raw}$: $\text{grad}*\phi_{iStefan} \sim |\text{grad}\alpha| * (U_{stef} * n_{Hat}) * S_f \sim \dot{m}_{Raw} / \rho_{int}$, which is the **interface-localized** mass-conservation contribution.

The commented block, if uncommented, would double-correct. The current $\text{divU} = \text{PCV}$ injection is the right choice. Delete the comment.

8. pEqn / continuity audit

pEqn.H:43-48:

```
fvcScalarMatrix p_rghEqn
(
    fvm::laplacian(rAUf, p_rgh)
    ==
    fvc::div(phiHbyA) - phaseChangePtr->PCV()
);
```

8.1 PCV correctness

$\text{PCV} = (Q_{pc}/h_{lv}) * (1/\rho_v - 1/\rho_l) = \dot{m}_{Raw} * (1/\rho_v - 1/\rho_l) > 0$ for evaporation. This is the correct volumetric source from mass-flux density difference. Sign convention matches $\text{grad}*U = \text{PCV}$. OK

8.2 phiStefan not in phiHbyA

ϕ_{iHbyA} does **not** include $\phi_{iStefan}$. The Stefan flux only enters alphaEqn. The dilatation enters here via PCV . This is the standard diffuse-interface decomposition (see Welch & Wilson 2000, Sato & Niceno 2013) and is defensible. The pressure field accommodates the volumetric source; the interface motion is carried by the kinematic $\phi_{iStefan}$ in alphaEqn. Document this explicitly in Sec.3.2 of the paper.

8.3 Missing Rhie-Chow consistency check

If any future change adds a body force to UEqn (e.g. EHD coupling for the user's broader research goal), that body force must also be added to phiHbyA. Add a TODO comment at the phig assembly to remind future developers.

9. Empirical regularization audit (full inventory)

Location	Mechanism	Defensibility	Recommendation
HardtWondra.C:283 alpha smoothing (nSmoothIter passes)	Pseudo-diffusion of alpha for gradient denoising	Defensible (Hardt & Wondra 2008); document Fo_per_iter cap	Keep, document Fo cap
HardtWondra.C:307 alphaGeom extra smoothing pass	Additional smoothing for normals only	Redundant with alphaSmooth	Merge into single field
HardtWondra.C:359-367 AiMax = 1.5/h_ref	Hard clip on	gradalpha	
HardtWondra.C:425-434 alphaThermal = 1/2(1+tanh 4(alpha-1/2))	tanh sharpening for thermal kEff	Defensible (sharpen thermal interface)	Document; eliminate harmonicBias_
HardtWondra.C:188 harmonicBias_ = 0.3	Bias toward liquid in qn kEff	Indefensible – duplicate kEff definition (Sec.4.4)	Remove
HardtWondra.C:486 TSense = max(T, Tsat)	One-sided T clip	Dead code (TSense unused for qn)	Remove
HardtWondra.C:506 TSense diffusion (D_Tsense)	Smooth TSense	Dead code	Remove
HardtWondra.C:561-578 evapSwitch = 1/2(1+tanh((T-Tsat)/0.25))	Sign-of-(T-Tsat) softening	Only used to gate Q_pc_thermal; magic DeltaT=0.25K	Defensible only if Q_pc_thermal kept; eliminate with Sec.4.3
HardtWondra.C:581-592 condSwitch	Cond branch sign	Dead – never used	Remove
HardtWondra.C:748 kinFloorCells_*h_ref denominator floor	Distance regularizer in qn search	Indefensible – biases qn down on fine meshes	Replace with LS gradient (Sec.4.3)
HardtWondra.C:782 qnSigned *= interfaceMask	pos(alpha-0.01)*pos(0.99-alpha) band mask	Defensible (excludes bulk)	Document threshold sensitivity
HardtWondra.C:919 mdot_ = clip(+/-mdotMax)	Hard clip on Helmholtz mdot	Backstop only	Add warning when triggered
HardtWondra.C:925 mdot_ *= RelaxFac_	Under-relaxation	Defensible	Keep
HardtWondra.C:982-990 nHatSmooth (2 Laplacian passes)	Normal denoising	Defensible	Keep, document
HardtWondra.C:1016-1021 AiFloor_kin = 5/m (absolute)	Division-safety in mdotRaw/Ai	Absolute floor, no mesh dependence – good	Keep

Location	Mechanism	Defensibility	Recommendation
HardtWondra.C:1066-1074 interfaceMaskF at faces	Suppress phiStefan in bulk faces	Defensible (zero-suppression in bulk)	Keep
HardtWondra.C:1133-1139 for(i=0;i<0;...) Q_pc smoothing	Loop count 0 – dead	Dead code	Remove
HardtWondra.C:205,1287 betaThermal_ parameter	Read from dict, never used	Dead parameter	Remove from .H/.C/dict
TEqn.H:36-43 Acoeff_floor (anti-trench)	Stiff penalty pulling T->Tsat (cold side)	Numerical bandaid (Sec.5.2 F3)	Disable by default; diagnostic only
TEqn.H:76-84 Acoeff_hot_floor (anti-superheat)	Stiff penalty pulling T->Tsat (hot side, cold-phase gate)	Numerical bandaid (Sec.5.2 F3)	Disable by default
TEqn.H:51-58 Commented-out coldSideGate block	Dead comment	Remove	

Total dead code to remove: TSense + D_Tsense + TSense smoothing loop + fvc::grad(TSense) + condSwitch + Q_pc smoothing loop (count=0) + betaThermal + commented-out coldSideGate block + harmonicBias_ (and dictionary read) + dead “STEP 10/11” duplicate comment blocks. Conservative estimate: ~120 lines of dead or stale code in HardtWondra.C alone.

10. Numerical concerns (prioritized)

10.1 Critical (publication-blocking)

- **C1.** Helmholtz inconsistency: Q_pc uses mdotRaw, Qcorr uses mdot (Sec.3). Pick one consistent variant.
- **C2.** qn reconstruction is first-order, single-neighbor, with a mesh-dependent floor that systematically under-predicts gradient (Sec.4.2.2 W3/W4). Replace with LS hot-side gradient (Sec.4.3).
- **C3.** Two latent fields Q_pc_ and Q_pc_thermal_ differing only in a magic denominator floor lack physical interpretation (Sec.4.2.2 W8). Collapse to one.
- **C4.** Acoeff_floor and Acoeff_hot_floor are stiffness penalties hiding upstream inconsistency. Their existence is evidence of an unstable qn (Sec.5.2 F3). Remove after fixing Sec.4.

10.2 High impact

- **H1.** phaseChangePtr->correct() runs only on firstIter. Energy equation cannot converge T-source feedback within an outer step (Sec.6.1).
- **H2.** Duplicate kEff definitions (Sec.4.4). Unify.
- **H3.** alpha1Gen() is dead infrastructure (Sec.7.1). Remove or document.
- **H4.** Dictionary-lifetime hack (pcDictPtr at outer scope) for one flag (Sec.6.4). Move flag into model base class.

10.3 Medium impact

- **M1.** Inconsistent naming phiTotal vs phiCN in alphaEqn (Sec.7.3).
- **M2.** Redundant rho/rhoCp rebuild between alphaEqn and post-pEqn (Sec.6.3).
- **M3.** F5 hidden conservation drift via rhoCpPhi (Sec.5.2 F5). Add consistency monitor.

- **M4.** Anisotropic $h_{\text{ref}} = \min(V)/\max(S_f)$ correctly handles thin cells but is global; for unstructured meshes, local $\text{mesh.V()}/\text{mesh.surfaceArea()}$ per-cell would be more appropriate. Defer to future work.

10.4 Low impact / cleanup

- **L1-L10.** Dead code removal (Sec.9 inventory).

11. Recommended implementation roadmap

Phase 1 – Cleanup (no physics change, ~1 day). - Delete dead code per Sec.9 (TSense block, condSwitch, betaThermal, count-0 loops, commented gates, stale STEP duplicates). - Unify `phiTotal/phiCN` naming in `alphaEqn`. - Remove duplicate `rho/rhoCp` rebuild. - Move `coldPhaseIsHighAlpha1` into `thermalPhaseChangeModel` base class. - Add `Q_pc_thermal_` removal / unification with `Q_pc_`.

Phase 2 – Helmholtz decision (1 day). - Decide localized vs redistributed HW (Sec.3). - If localized: remove `mdot_` Helmholtz solve, set `Qcorr == 0`, rename diagnostics. - If redistributed: switch `phiStefan` and PCV to use `mdot`. Document that this restores 25 % interface-position error; case parameters must be re-validated.

Phase 3 – qn reconstruction (2-3 days). - Implement LS hot-side gradient (Sec.4.3). - Unify `kEff` field (Sec.4.4). - Remove `kinFloorCells_`, `AiFloor_kin` may stay (absolute, defensible). - Re-run Sheikh 1D Stefan validation.

Phase 4 – TEqn simplification (1 day). - Move `Acoeff_floor` and `Acoeff_hot_floor` to dictionary-controlled diagnostics (off by default). - Remove `Q_cond` split if pure evaporation cases (or keep for generality with clear documentation). - Replace `Acoeff` form with single explicit RHS option for ablation study.

Phase 5 – Convergence study (2 days). - Per-outer-iter `correct()` call (Sec.6.1). - Conservation monitor function-object (Sec.5.2 F5). - Mesh convergence with new qn reconstruction.

Phase 6 – Documentation (continuous). - Inline derivation of all coefficients. - Document any remaining magic numbers with citations. - Rebuild user manual to match cleaned solver.

12. Summary of recommended state after audit

Field inventory after cleanup: - **HardtWondra.C:** ~3-pathway architecture preserved exactly; ~150 lines removed; one unified `kEff`, one unified qn (LS), one `mdotRaw`, one `Q_pc = mdotRaw*h_lv`, one `phiStefan`, optional Helmholtz `mdot` retained only if `Qcorr` remains. - **TEqn.H:** ~30 lines (no `Acoeff` floors by default; single `Acoeff` from `Q_pc`); diagnostic flag for emergency floors. - **alphaSuSp.H:** trivial `divU=PCV`-only injection; `Su=0` explicitly documented. - **createFields.H:** `pcDictPtr` removed; flag accessed via model base class. - **interTempFoam.C:** optional per-outer-iter `correct()` flag; consistent `rho/rhoCp` rebuild point.

Result: smaller, more defensible, faster, and publishable.

13. Defence of smoothing – what is and isn’t justifiable

The audit flags many smoothing operations as suspect. Before recommending cleanup, each must be examined on its merits. The criterion is single and sharp:

A regularizer is **publication-defensible** if and only if its characteristic length scale is set by mesh geometry alone, vanishes as $h \rightarrow 0$, and discretizes a continuum operator that has an honest place in the diffuse-interface Stefan formulation.

This is the *geometric-dependence* criterion. Anything tied to Deltat, to the solver iteration count, or to a tuned constant with no derivation, fails.

13.1 Why a diffuse-interface Stefan solver needs *some* smoothing

The Hardt-Wondra formulation reinterprets the sharp-interface Stefan condition

$$\rho_l v_n h_{lv} = k \frac{dT}{dn}|_{\Gamma} \quad (\text{sharp})$$

as a volumetric source over the band where $|\text{grad}\alpha| \neq 0$:

$$\dot{m}(x) = |\text{grad}\alpha_s|(x) * q_n(x) / h_{lv} \quad (\text{diffuse})$$

Three field reconstructions are *required* in this formulation, and each demands a band-limiting filter:

Reconstructed field	Why band-limiting is required
$ \text{grad}\alpha_s $ (interface area density)	Raw MULES α has a Heaviside-like staircase across the interface band. $\text{grad}\alpha$ then contains $O(1/h)$ spikes that do not converge under mesh refinement and violate the asymptotic identity $\int \text{grad}\alpha dV \approx A_{\Gamma}$.
$\hat{n} = \text{grad}\alpha / \text{grad}\alpha $ (interface normal)	The normal is undefined in bulk and rotates wildly on the band boundaries. Without filtering, $\phi_{\text{Stefan}} = U_s * (\hat{n} * S_f)$ injects rotational components that distort the interface.
q_n and $\dot{m}_{\text{raw}}$ (Stefan flux & raw mass source)	Conductive flux reconstructed across the diffuse band has sub-cell variation that the energy equation cannot resolve. Without a controlled spreading of the latent sink, T develops one-cell spikes that propagate through A_{coeff} back into q_n – a divergent feedback.

So three filtering operations are physically necessary. The question is *how many distinct smoothing fields and operators are needed*, not whether to smooth at all.

13.2 The diffusion-of-field-variable identity

Every smoothing operation in the current solver implements **forward-Euler heat diffusion with a fictitious time τ** :

$$\partial \alpha_s / \partial \tau = D \nabla^2 \alpha_s, \quad \alpha_s(\tau=0) = \alpha$$

Iterated over N explicit steps of length $\Delta\tau$. The continuum Green's function gives

$$\alpha_s(x, \tau) = \int G(x-x', \tau) \alpha(x') dx', \quad G = (4\pi D\tau)^{-d/2} \exp(-|x|^2/4D\tau)$$

A Gaussian filter of width

$$\sigma(\tau) = \sqrt{2 D \tau} \quad (1-D)$$

In the code:

```
D_smooth = Fo_per_iter * h_ref^2 / Deltat // [m^2/s]
alpha_s <- alpha_s + D_smooth * grad^2alpha_s * Deltat // explicit step
```

Net width after N iterations:

$$\begin{aligned} \sigma^2 &= 2 * D_{\text{smooth}} * (N * \Delta\tau) \\ &= 2 * (F_{\text{o_per_iter}} * h_{\text{ref}}^2 / \Delta\tau) * (N * \Delta\tau) \\ &= 2 * F_{\text{o_per_iter}} * N * h_{\text{ref}}^2 \end{aligned}$$

The Deltat cancels exactly. σ depends only on h_{ref} and N . With the code's $\text{Fo_per_iter} = \alpha\text{SmoothWidth}^2/N$ and the cap $\text{Fo_per_iter} \leq 0.25$:

$$\sigma = h_{\text{ref}} * \sqrt{2 * \alpha\text{SmoothWidth}^2 / N * N} = \alpha\text{SmoothWidth} * h_{\text{ref}} * \sqrt{2}$$

So the filter width is **purely geometric**: $\sigma = O(h_{\text{ref}})$. This is the asymptotic property that lets the diffuse-interface formulation converge to the sharp Stefan limit as the mesh is refined.

This identity is the foundation of the cleanup proposal: **every smoothing operation in HW *should* be geometric-only. Any that isn't is a bug.**

13.3 Per-operation defence

Smoothing	Operator	Length scale	Deltat-dep?	Defensible?
$\alpha \rightarrow \alpha_s$ ($n\text{SmoothIter}$ passes)	Explicit $d\alpha/d\tau =$ $D\text{grad}^2\alpha$	$\sigma \sim$ $\alpha\text{SmoothWidth}$ $* h_{\text{ref}}$	No (cancels)	Yes – required for $\backslash \text{grad}\alpha \backslash$ denoising
$\alpha_s \rightarrow$ αGeom (one more pass)	Same operator	σ adds $\sim h_{\text{ref}}/\sqrt{N}$	No	Redundant – collapse into α_s
$T \rightarrow T_{\text{sense}}$ clip + Lap	Same operator	$\sigma \sim$ $0.55 * h_{\text{ref}}$	No	Dead – T_{sense} unused for qn
$n\text{Hat} \rightarrow$ $n\text{Hat}_{\text{smooth}}$ (2 passes, no Deltat)	Direct $n +$ $\lambda^2 \text{grad}^2 n$	$\sigma \sim$ $\sqrt{2}\lambda =$ $\sqrt{2}\lambda \text{SmearCells} * h_{\text{ref}}$	No	Yes – required for ϕ Stefan stability, geometric
$\text{mdot}_{\text{raw}} \rightarrow \text{mdot}$ Helmholtz	Implicit $(1 - \lambda^2 \text{grad}^2) \text{mdot}$ $= \text{mdot}_{\text{raw}}$	$\sigma_{\text{out}}^2 =$ $\sigma_{\text{in}}^2 +$ λ^2	No	Conditionally yes – see Sec.3; load-bearing only if $Q_{\text{pc}} = \text{mdot} * h_{\text{lv}}$
Q_{pc} extra Laplacian (count = 0)	Dead	n/a	n/a	Dead
TEqn $\text{Acoeff}_{\text{floor}}$	Implicit penalty	$\rho C_p / \text{Deltat}$ magnitude	YES	NO – only Deltat-dependent regularizer in the entire code
TEqn $\text{Acoeff}_{\text{hot}_{\text{floor}}}$	Implicit penalty	$\rho C_p / \text{Deltat}$ magnitude	YES	NO – same
qn search distance floor $\text{kinFloorCells} * h_{\text{ref}}$	Hard floor in denominator	$\text{kinFloorCells} *$ h_{ref}	No	Geometric but biases qn downward – replace via LS gradient (Sec.4.3)

Verdict: of nine “smoothing/limiting” mechanisms inspected: - 2 are physically required and properly mesh-set (α -smoothing, $n\text{Hat}$ -smoothing). - 1 is conditionally required (Helmholtz, depends on Sec.3 choice). - 4 are dead or redundant (αGeom 2nd pass, T_{sense} block, Q_{pc} smoothing, qn denominator floor). - **2 are Deltat-dependent and indefensible** – $\text{Acoeff}_{\text{floor}}$, $\text{Acoeff}_{\text{hot}_{\text{floor}}}$.

The “bloat” perception is correct in count but not in spirit: the *architecture* is sound (every operation has a defensible continuum analogue); the *implementation* duplicates, applies dead operations, and adds two Deltat-dependent penalties that the rest of the model is structured to avoid.

14. Why the current Deltat-dependent terms exist (and how to eliminate them)

$A_{coeff_floor} = C_{floor} * \rho C_p / \Delta t * 4\alpha(1-\alpha) * \max(T_{sat} - T, 0)$ provides an implicit pull $T \rightarrow T_{sat}$ with characteristic time $\Delta t / C_{floor}$. As $\Delta t \rightarrow 0$ the penalty becomes infinitely stiff – pinning T to T_{sat} in the diffuse band exactly. This is the *only* way to absorb spurious overshoots arising from upstream q_n noise without changing other model components.

The root cause of those overshoots is the q_n reconstruction (Sec.4) – a single-neighbor first-order one-sided difference that switches stencil between time steps as the interface moves, producing sub-cell Δq_n discontinuities that release latent heat at non-physical rates. The $\rho C_p / \Delta t$ floor is then *not* a physics term – it is an implicit limiter on the *temperature jump* per timestep. This is the fingerprint of a stiff underlying source.

Eliminating Deltat dependence requires fixing the upstream q_n , not adding more downstream regularizers. With the LS hot-side gradient (Sec.4.3): - q_n is continuous in time as the interface moves (LS weights vary smoothly). - dq_n/dt is bounded by mesh geometry alone. - T overshoots in the diffuse band reduce to truncation error of the Laplacian discretization, controllable by mesh refinement. - The A_{coeff} term alone, with $Q_{pc}/(T-T_{sat})$ stiffness coefficient, is sufficient to enforce T_{sat} in the diffuse band. - A_{coeff_floor} and $A_{coeff_hot_floor}$ can be removed.

This converts the solver from “stabilization-by-stiffness” to “stabilization-by-physical-consistency”. The former requires parameter tuning per case; the latter does not.

15. Clean Stefan solver architecture (proposed)

15.1 Design principles

1. **One filtered alpha field.** All geometric reconstructions ($|\nabla \alpha|$, n_{hat} , $interfaceArea$, $masks$) come from a single smoothed α with band width $\delta = N_{band} * h_{ref}$ (default $N_{band} = 1.5$).
2. **One filter operator.** A single implicit Helmholtz smoother $(I - \delta^2/2 * \nabla^2) \alpha_s = \alpha$ replaces multi-pass explicit smoothing. Equivalent in steady state to $N \rightarrow \infty$ explicit passes (variance $\sigma^2 = \delta^2$), no stability restriction, no τ , no Δt anywhere.
3. **One length scale delta.** The same δ controls: α band, n_{hat} filter width, Helmholtz redistribution length (if retained), q_n -search regularizer. Mesh-refinement convergence is then governed by a single parameter.
4. **Zero Deltat-dependent regularizers.** Every implicit penalty, every stabilizer, every floor, scales with h_{ref} or its derivatives only.
5. **Single q_n pathway.** No thermal/kinematic split. One LS-reconstructed $dT/dn|_{hot}$, one q_n , one $\dot{m}_{raw}$. Q_{pc} , ϕ_{Stefan} , PCV derived from this single chain.

15.2 Mathematical structure

Define the band length:

$$\delta = N_{band} * h_{ref}, \quad N_{band} \in [1, 2], \quad \text{default } 1.5$$

Filtered phase field:

$$\alpha_s = (I - (\delta^2/2) \nabla^2)^{-1} \alpha \quad (\text{one implicit elliptic solve})$$

Equivalently α_s is the Gaussian filter of α with variance δ^2 , by identity between Helmholtz smoothing and Gaussian convolution in the continuum limit (operator splitting in radial Fourier space gives $\alpha_s = \alpha / (1 + \delta^2 k^2/2) \sim \exp(-\delta^2 k^2/2)$ for $\delta k \ll 1$).

Geometric primitives (all from α_s):

$$A_i = |\nabla \alpha_s| \quad [1/m]$$

$n_{\text{Hat}} = \text{gradalpha}_s / (|\text{gradalpha}_s| + \text{eps_alpha})$ $\text{eps_alpha} = \text{SMALL} * 1/m$
 $n_{\text{Hat_f}} = (\text{filtered}) \text{ face interpolation}$ no extra smoothing needed; α_s already filtered

Hot-side conductive gradient (LS, Sec.4.3):

$M_c = \text{Sigma}_n w_n * (x_n - x_c)(x_n - x_c)T$
 $b_c = \text{Sigma}_n w_n * (T_n - T_{\text{sat}}) * (x_n - x_c)$
 $w_n = \max(T_n - T_{\text{sat}}, 0)$ one-sided weight
 $(\text{gradT_hot})_c = M_c^{-1} * b_c$ (Moore-Penrose if rank-deficient)
 $dT/dn|_{\text{hot}} = \max((\text{gradT_hot} * n_{\text{Hat}}), 0)$

Effective interface conductivity (single field, hot-side biased):

$k_{\text{eff}} = k_{\text{hot}} * \alpha_{\text{hot}} + k_{\text{cold}} * (1 - \alpha_{\text{hot}})$ arithmetic

where $\alpha_{\text{hot}} = \alpha_s$ if cold phase has $\alpha=0$, $1-\alpha_s$ otherwise. Use arithmetic, not harmonic
 – the Stefan flux is being *delivered* by the hot phase, not transmitted through a series resistance.

Single Stefan flux chain:

$q_n = k_{\text{eff}} * dT/dn|_{\text{hot}}$ [W/m²]
 $\text{mdot_raw} = A_i * q_n / h_{\text{lv}}$ [kg/m³/s]
 $\text{mdot} = (I - \text{delta}^2 \text{grad}^2)^{-1} \text{mdot_raw}$ (optional; only if Qcorr retained)
 $Q_{\text{pc}} = \text{mdot_raw} * h_{\text{lv}}$ (localized, default)
 $\text{PCV} = \text{mdot_raw} * (1/\rho_v - 1/\rho_l)$
 $U_s = \text{mdot_raw} / (A_i + \text{eps}_A) / \rho_{\text{int}}$
 $\text{eps}_A = \text{SMALL} * \max(A_i)$ pure zero-guard, mesh-independent in magnitude
 $\text{phiStefan} = (U_s n_{\text{Hat_f}}) * S_f$

Energy equation (no floors, no condSwitch, no Q_{cond} split):

$d(\rho C_p T)/dt + \text{grad}*(\rho C_p \text{phi } T) - \text{grad}*(k_{\text{eff}}, f \text{ grad}T)$
 $+ \text{Sp}(A_c, T) = A_c * T_{\text{sat}}$
 $A_c = \max(Q_{\text{pc}} / \max(T - T_{\text{sat}}, \text{eps}_T), 0)$ $\text{eps}_T = 0(0.1) \text{ K}$

The Acoeff term alone is sufficient: in cells with $Q_{\text{pc}} \neq 0$, T is pulled toward T_{sat} with characteristic rate $A_c / \rho C_p$. There is no $\rho C_p/\text{Deltat}$ penalty anywhere. Stability is achieved by physical consistency, not by stiffness.

15.3 Why no Deltat appears anywhere

Trace every length scale:

Field	Length scale	Source
α_s band sigma	$\text{delta} = N_{\text{band}} * h_{\text{ref}}$	mesh
n_{Hat} filter	delta (same)	mesh
mdot Helmholtz lambda	delta (same)	mesh
qn LS stencil	cellCells extent $\sim h_{\text{ref}}$	mesh
A_c stiffness	$Q_{\text{pc}}/(T-T_{\text{sat}})$ – depends on flow state, not Deltat	physics
Anti-trench floor	REMOVED	–
Anti-superheat floor	REMOVED	–
AiFloor in U_s	absolute, $\text{eps} * \max(A_i)$	mesh-relative

Compare to the current code, which has $C_{\text{floor}} * \rho C_p/\text{Deltat}$ in two places. After cleanup: zero Deltat in any source/sink coefficient. The only Deltat remaining is in $\rho C_p * dT/dt$, which is correct.

15.4 Asymptotic limit $h \rightarrow 0$

- $\delta \rightarrow 0 \Rightarrow \alpha_s \rightarrow \alpha$ (no smoothing), $|\text{grad}\alpha| \rightarrow \delta_\Gamma$ (surface δ).
- $A_c \rightarrow \infty$ on a measure-zero set $\Rightarrow T = T_{\text{sat}}$ on the interface (Dirichlet).
- $\dot{m}_{\text{raw}} \rightarrow A_i \cdot q_n / h_{lv} \Rightarrow \int \dot{m} dV \rightarrow \int_\Gamma \dot{m} dA$ (sharp Stefan condition).
- $\phi_{\text{Stefan}} \rightarrow$ sharp jump in velocity at the interface.

This is the formal sharp-limit recovery that publication-grade Stefan formulations must demonstrate. The current code with Δt -dependent floors **does not** have this limit: as $h \rightarrow 0$ with Δt fixed, the floors remain at fixed strength while the conductive gradient becomes sharper, so the floor becomes increasingly irrelevant – but as $h \rightarrow 0$ and $\Delta t \rightarrow 0$ (CFL-locked), the floors become infinitely stiff. The behavior depends on which limit is taken first, which is a publication red flag.

15.5 Implementation footprint

The clean architecture replaces ~1300 lines of `HardtWondra.C` with ~400. Pseudocode skeleton:

```
void HardtWondra::correct()
{
    const scalar h = computeHref();
    const dimensionedScalar deltaSqr_half("d2h", dimArea, 0.5*sqr(N_band_*h));
    const dimensionedScalar deltaSqr      ("d2",   dimArea,      sqr(N_band_*h));

    // 1. ONE filter on alpha (implicit, no Deltat)
    volScalarField alphaS = alpha1_;
    fvScalarMatrix sEqn
    (
        fvm::Sp(dimensionedScalar("one", dimless, 1.0), alphaS)
        - fvm::laplacian(deltaSqr_half, alphaS)
        == alpha1_
    );
    sEqn.solve();
    alphaS.clip(0, 1);

    // 2. Geometric primitives -- all from alphaS
    volVectorField gAlpha = fvc::grad(alphaS);
    interfaceArea_ = mag(gAlpha);
    volVectorField nHat = gAlpha / (interfaceArea_ + epsA_);

    // 3. LS hot-side gradient -> qn (Sec.4.3)
    volScalarField dTdn = leastSquaresHotGradient(T_, T_sat_, nHat);
    volScalarField kEff = k_hot_*alphaHot_ + k_cold_*(1-alphaHot_); // arithmetic
    qn_ = kEff * dTdn;
    mdotRaw_ = interfaceArea_ * qn_ / h_lv_;

    // 4. (Optional) Helmholtz redistribution -- only if useEnthalpyCorrection
    if (useEnthalpyCorrection_)
    {
        fvScalarMatrix mEqn
        (
            fvm::Sp(dimensionedScalar("one", dimless, 1.0), mdot_)
            - fvm::laplacian(deltaSqr, mdot_) == mdotRaw_
        );
        mEqn.solve();
        Qcorr_ = -mdot_*(cp1_-cp2_)*(T_-T_sat_);
        Q_pc_ = mdot_*h_lv_; // consistent HW
    }
}
```

```

    }
    else
    {
        Qcorr_ = dimensionedScalar(Qcorr_.dimensions(), 0);
        Q_pc_ = mdotRaw_*h_lv_; // localized
    }

    // 5. Stefan flux (kinematic; mesh-set epsA)
    volScalarField Ustef = mdotRaw_/(interfaceArea_ + epsA_)
        / rhoInt(alphaS);
    phiStefan_ = fvc::interpolate(Ustef * nHat) & mesh_.Sf();
}

```

One smoothing solve. One LS gradient. One qn. One mdot chain. No dead code, no tuned constants beyond N_{band} (geometric).

16. Generalization to arbitrary phase change

The cleaned formulation is symmetric in melting/evaporation/solidification/ condensation provided three accessors are added to the model:

```

class thermalPhaseChangeModel
{
public:
    virtual bool    coldPhaseIsHighAlpha1() const = 0; // Sec.6.4 fix
    virtual scalar  h_lv_signed() const = 0; // h_lv for evap, +Lf for melt
    virtual scalar  densityRatioFactor() const = 0; // (1/rho_cold - 1/rho_hot)
    ...
};

```

Then: - **Evaporation** (water boiling): cold=liquid ($\alpha_1=1$), $\text{densityRatioFactor} = 1/\rho_v - 1/\rho_l > 0$, vapor created $\rightarrow \text{PCV} > 0$. - **Condensation**: same signs reversed via $dT/dn|_{\text{hot}} < 0$ impossibility (no hot neighbor with $T < T_{\text{sat}}$) – model returns $\text{mdot}=0$; condensation case uses cold-side LS gradient instead. Add $dTdn_{\text{cold}}$ accessor for bidirectional models. - **Melting** (Sheikh Stefan): cold=solid ($\alpha_1=1$), $\text{densityRatioFactor} = 1/\rho_l - 1/\rho_s$ (often small), liquid produced from solid. - **Solidification**: hot-side flag inverts.

The single LS stencil + sign convention via $\text{densityRatioFactor}$ handles all four regimes with no branching in the inner loop. The coldSideGate , $\text{coldPhaseIsHighAlpha1}$, and Q_{cond} split currently in the solver can all be eliminated by routing through these three accessors.

17. Summary – the defensible minimum

A publication-grade Stefan VOF solver retains, with proper derivation:

1. **One implicit alpha-filter**: $(I - \delta^2/2 \text{ grad}^2)\alpha_s = \alpha$, $\delta = N_{\text{band}} * h_{\text{ref}}$.
2. **One geometric reconstruction**: $A_i = |\text{grad}\alpha_s|$, $n_{\text{Hat}} = \text{grad}\alpha_s / (|\text{grad}\alpha_s| + \epsilon)$.
3. **One LS hot-side gradient** for qn, no magic floors.
4. **One arithmetic hot-biased kEff** for qn (sharp Stefan delivery side).
5. **One harmonic kappaf** for TEqn Laplacian (series resistance through the diffuse band).
6. **One Helmholtz redistribution** for mdot – only if $Q_{\text{pc}} = \text{mdot} * h_{\text{lv}}$ with Qcorr enabled. Otherwise drop entirely.
7. **One Acoeff** in TEqn: $Q_{\text{pc}}/\max(T - T_{\text{sat}}, \epsilon_{\text{T}})$. No floors.
8. **One length scale delta** governing all filters.

9. **Zero Deltat-dependent coefficients** anywhere outside d/dt operators.
10. **One phase-change correction call per outer iteration** (not just firstIter – Sec.6.1).

Everything else is bloat or bandaid. The current solver contains the correct physics buried inside this scaffolding; cleanup is removal, not rewrite.

18. Directionality analysis of the phiStefan vector

Construction chain (HardtWondra.C):

```
gradAlpha = grad(alphaGeom)           // alphaGeom = doubly-smoothed alpha1
nHat      = gradAlpha / |gradAlpha|    // unit cell-centered normal
nHatSmooth = 2 Laplacian passes, then renormalize
Ustef     = mdotInterfacial / rhoInt   // scalar, >= 0 by construction
phiStefan = Ustef_f * (nHatSmooth_f . Sf) * interfaceMaskF
```

18.1 Direction of nHat

$\mathbf{nHat} = \mathbf{grad}(\alpha_1) / |\mathbf{grad}(\alpha_1)|$ points from the $\alpha=0$ phase toward the $\alpha=1$ phase. Whether this is “hot->cold” or “cold->hot” depends on which phase α_1 represents:

Case	$\alpha_1=1$	$\alpha_1=0$	$\mathbf{nHat}$ points
Melting (coldPhaseIsHighAlpha1=true)	solid (cold)	liquid (hot)	hot -> cold
Evap, liquid superheated	liquid (hot)	vapor (cold)	cold -> hot
Evap, vapor superheated	liquid (cold)	vapor (hot)	hot -> cold

The `coldPhaseIsHighAlpha1` flag (TEqn.H) is consulted only by the anti-superheat gate – the `phiStefan` construction never reads it. The direction is set entirely by the local $\mathbf{grad}(\alpha)$, which is correct: direction is a property of the α field, not a runtime choice.

18.2 Sign of Ustef

```
mdotRaw = Ai * qn / h_lv,    qn = kEff * dTdn_hot
Ustef    = mdotRaw / [(Ai + AiFloor_kin) * rho_int]
```

The neighbor search at `HardtWondra.C:740` enforces `superheat = T_nbr - Tsat > 1e-6`, so `dTdn_hot >= 0` always. Therefore `qn >= 0`, `mdotRaw >= 0`, `Ustef >= 0`.

Implication. The solver is **uni-directional**. It models evaporation and melting (forward phase change, hot phase consumed) but **cannot represent condensation or solidification** – `Ustef` cannot flip sign. `Q_cond = min(Q_pc, 0)` and `condSwitch` are dead branches in any case using this construction.

18.3 Direction of interface motion

Stefan contribution to $d\alpha_1/dt + \text{div}(\alpha_1 (\mathbf{\phi} + \mathbf{\phiStefan})) = 0$:

$$\begin{aligned} \text{div}(\alpha_1 * \mathbf{Ustef} * \mathbf{nHat}) &\sim \mathbf{Ustef} * (\mathbf{nHat} \cdot \mathbf{grad}(\alpha_1)) \\ &= \mathbf{Ustef} * |\mathbf{grad}(\alpha_1)| \quad (\mathbf{nHat} \text{ aligns with } \mathbf{grad}) \\ &\geq 0 \end{aligned}$$

So $d\alpha_1/dt \leq 0$ in the band: $\alpha=1$ phase always **shrinks**, $\alpha=0$ phase grows, interface translates **in the $+\mathbf{nHat}$ direction**.

- Melting: $\mathbf{nHat}$ hot->cold $\Rightarrow$ interface into solid; solid $\alpha_1=1$ shrinks. **OK.**
- Evap (liquid hot): $\mathbf{nHat}$ cold->hot $\Rightarrow$ interface into liquid; liquid $\alpha_1=1$ shrinks. **OK.**

Sign-consistency across alpha conventions is automatic because $U_{stef} \geq 0$ and n_{Hat} is built from $\text{grad}(\alpha_1)$. Direction is not user-chosen – it is forced by the sign of the hot-side conductive flux.

18.4 Face-orientation convention

$\text{phiStefan}_f = U_{stef}_f * (n_{Hat}_f \cdot Sf)$, Sf points owner $\rightarrow$ neighbor

- positive $\Rightarrow$ mass-tracking flux owner $\rightarrow$ neighbor (interface advancing owner-to-neighbor in alpha-transport)
- negative $\Rightarrow$ opposite

Standard OpenFOAM face-flux convention; no MULES handling needed.

18.5 Directional weaknesses

D1. Triple smoothing of normal direction. n_{Hat} is built from α_{Geom} (already smoothed twice from α_1) and then smoothed by two additional Laplacian passes and renormalized. Each pass averages normal directions across neighbors. On curved interfaces this rotates n_{Hat}_f away from the true geometric normal by $O(\text{curvature} * \text{delta_band})$.

D2. Face-interpolation of a unit vector underestimates magnitude. $\text{fvc}::\text{interpolate}(n_{HatSmooth})$ produces a face vector with

$$|n_{Hat}_f| = |0.5 (n_{Hat}_0 + n_{Hat}_N)| = \cos(\theta/2)$$

where θ is the angle between cell-centered normals. phiStefan magnitude is multiplied by $\cos(\theta/2)$, under-predicting interface speed at curvature. For $\theta=60^\circ$ this is 13 % error. For 1D planar fronts (Sheikh validation) $\theta=0$ and the error vanishes – which is why this defect has not surfaced.

Mitigation. Decouple magnitude from direction at faces:

```
const surfaceVectorField nHatf_raw = fvc::interpolate(nHatSmooth);
const surfaceVectorField nHatf
(
    nHatf_raw
    / max(mag(nHatf_raw),
          dimensionedScalar("eps", dimless, SMALL))
);
phiStefan_ = fvc::interpolate(Ustef) * (nHatf & mesh_.Sf()) * interfaceMaskF;
```

One extra mag + divide per face, no extra solve.

D3. interfaceMaskF fixed threshold. $\text{interfaceMaskF} = \min(\min(\alpha_f, 1-\alpha_f)/0.05, 1)$ uses a hard cutoff at $\alpha=0.05$. At thin diffuse interfaces, only one face per interface satisfies α_f in $(0.05, 0.95)$, producing direction-dependent step changes in phiStefan magnitude along the interface tangent. Effect: small along-interface waviness uncorrelated with physics. **Recommendation:** replace with $1 - \tanh^2(2*(2*\alpha_f - 1))$, a smooth band centered at $\alpha=0.5$ matched to $\alpha_{SmoothWidth}$.

D4. Tangential leakage from smoothed n_{Hat} . Mathematically n is purely normal to the iso-alpha surface. After two Laplacian passes, $n_{HatSmooth}$ is no longer exactly aligned with $\text{grad}(\alpha)$ and inherits orientation from neighbors with possibly different gradients. $n_{Hat}_f \cdot Sf$ then includes tangential-motion contributions, producing spurious tangential interface drift. **Recommendation:** at the end of n_{Hat} smoothing, reconstruct n_{Hat} from the smoothed α_{Geom} directly:

```
nHatSmooth = grad(alphaGeom)
    / max(mag(grad(alphaGeom)),
          dimensionedScalar("eps", dimless/dimLength, SMALL));
```

Achieves smoothness via the alpha field; guarantees purely-normal output.

D5. Uni-directionality is hard-coded. The `superheat > 0` filter excludes cold neighbors entirely. Bidirectional capability requires either (i) a parallel `dTdn_cold` reconstruction with sign-flip when `dTdn_cold > dTdn_hot`, or (ii) the LS gradient of Sec.4.3 with signed weights $w_n = T_n - T_{sat}$ (which carries sign automatically). Option (ii) is the publication-grade path: one solve handles both regimes, `Ustef` sign follows naturally, `Q_cond` becomes live code.

18.6 Verdict

Property	Status
Sign of <code>Ustef</code>	Correct for forward phase change; structurally cannot reverse
Direction (alignment with <code>grad(alpha)</code>)	Correct, automatic across <code>alpha</code> conventions
Magnitude at curvature	Under-predicted by $\cos(\theta/2)$ at curved interfaces
Tangential purity	Degraded by <code>nHat</code> smoothing; restorable via <code>grad(alphaGeom)</code> reconstruction
Mask continuity	Degraded by hard <code>alpha</code> cutoff; restorable via smooth <code>tanh</code> band

For 1-D planar validation (Sheikh) all D1-D4 vanish; the construction is sound. For 2-D/3-D curved-interface targets, D2 and D4 should be fixed (5-line changes each) before mesh-convergence studies. D5 is a deliberate restriction; condensation/solidification scope requires the Sec.4.3 LS reformulation.

19. UEqn and pEqn implementation review

19.1 UEqn (UEqn.H)

```
fvVectorMatrix UEqn
(
    fvm::ddt(rho, U) + fvm::div(rhoPhi, U)
    + MRF.DDt(rho, U)
    + turbulence->divDevRhoReff(rho, U)
    == fvOptions(rho, U)
);
UEqn.relax();
fvOptions.constrain(UEqn);
if (pimple.momentumPredictor())
{
    solve(UEqn == fvc::reconstruct((
        mixture.surfaceTensionForce()
        - ghf*fvc::snGrad(rho)
        - fvc::snGrad(p_rgh)
    ))*mesh.magSf());
    fvOptions.correct(U);
}
```

This is **vanilla interFoam UEqn, unmodified**. Observations:

O1. No Stefan body force. Correct decision. The Stefan velocity is NOT a real momentum carrier – it is an interface-tracking auxiliary field. Injecting a `phiStefan`-derived body force into the mixture momentum equation would double-count the dilatation already handled by `pEqn`’s PCV source.

O2. No latent-heat body force. Correct. Volumetric expansion forces arising from phase change enter via PCV in the pressure equation ($\text{div}(U) = \text{PCV}$ constraint), which produces the correct pressure-driven outflow from the interface.

O3. rho is the volume-averaged mixture density. Built from $\alpha_1 \rho_1 + (1 - \alpha_1) \rho_2$ and updated *after* pEqn (interTempFoam.C:180-186). The momentum diffusion operator `turbulence->divDevRhoReff` uses the same alpha-weighted blending for viscosity. This is the standard immiscible-two-phase mixture model; no phase-change-specific modification needed.

O4. rhoPhi is updated in alphaEqn.H:273 after MULES. This is critical: after phase change, alpha has changed and rho has changed, so $\rho\phi = \alpha\phi_{10}(\rho_{1f} - \rho_{2f}) + \phi\rho_{2f}$ carries the post-phase-change momentum flux. **Correct ordering.**

Verdict on UEqn: Clean, no modifications required for HW phase change. Future EHD/Marangoni extensions inject as additional RHS `fvc::reconstruct(...)` terms; the surface-tension term placement is the canonical Rhie-Chow-compatible insertion site.

19.2 pEqn (pEqn.H)

```
rAU = 1.0/UEqn.A();
HbyA = constrainHbyA(rAU*UEqn.H(), U, p_rgh);
phiHbyA = fvc::flux(HbyA) + ddtCorr;
phig = (surfaceTensionForce() - ghf*snGrad(rho)) * rAUf * magSf;
phiHbyA += phig;

while (correctNonOrthogonal())
{
    fvScalarMatrix p_rghEqn
    (
        fvm::laplacian(rAUf, p_rgh)
        == fvc::div(phiHbyA) - phaseChangePtr->PCV() // <-- ONLY HW modification
    );
    p_rghEqn.solve(...);
    phi = phiHbyA - p_rghEqn.flux();
    U = HbyA + rAU*fvc::reconstruct((phig - p_rghEqn.flux())/rAUf);
}
```

The pEqn is **standard interFoam with a single modification**: the `- PCV()` source on the RHS.

Mathematical role of PCV in pEqn.

The pressure equation is $\text{div}(\mathbf{U}) = \text{source}$. For incompressible interFoam this source is identically zero. With phase change:

$$\text{div}(\mathbf{U}) = \text{mdot} * (1/\rho_v - 1/\rho_l) = \text{PCV}$$

This comes from summing the two phase-mass equations after dividing each by its constant density. $\text{PCV} > 0$ in evaporation: vapor is being created at lower density than the liquid it replaces, so the local fluid expands at rate PCV. The pressure correction step computes a `p_rgh` whose gradient produces a velocity correction such that the new face fluxes satisfy $\text{div}(\phi) = \text{PCV} * V_{\text{cell}}$. **Correct and necessary.**

Consistency observations.

P1. phiHbyA does NOT contain phiStefan. Stefan flux is an interface-tracking velocity, not a momentum-carrying velocity. Including it in `phiHbyA` would imply momentum flux at the interface from a non-physical velocity – a Rhie-Chow violation. **Correct exclusion.**

P2. phig does NOT contain a phase-change body-force component. This matches O2: no Stefan body force in UEqn, no Stefan flux in `phiHbyA`. The pair is Rhie-Chow-consistent.

P3. PCV uses Q_pc = mdotRaw*h_lv (localized). The PCV source is therefore concentrated in the diffuse interface band (1-2 cells wide). The pressure Laplacian then has a sharply-localized RHS, producing $O(1/h)$ pressure spikes at the interface. This is the physically-correct behavior (mass is being created at the

interface, not redistributed) but it strains the GAMG solver on fine meshes: tighter `tolerance` (1e-8 instead of 1e-7) and 2+ non-orthogonal correctors may be needed.

P4. Alternative formulation: Helmholtz-smoothed PCV. If `Q_pc = mdot*h_lv` (Sec.3 redistributed variant), PCV is spread over $\sim\lambda$. Pressure field is smoother, easier to solve, but introduces phantom volume sources in cells outside the physical phase-change zone – the same artifact `Qcorr` is designed to remove from the energy equation. **Consistency rule:** if `Q_pc` uses `mdotRaw` (localized), PCV also uses `mdotRaw` via the same `Q_pc` accessor. Current code does this correctly. If `Q_pc` switches to `mdot`, PCV follows automatically (same accessor) and `Qcorr` must be enabled.

P5. `p_rghEqn.flux()` is used to update `phi`. This is the standard Rhie-Chow-consistent flux update. Phase change does not alter this step – the PCV source enters the pressure Poisson, the resulting flux correction propagates to `phi`, and `phi` is automatically divergence-balanced with PCV at the discrete level.

Verdict on `pEqn`: Mathematically correct and minimally modified from vanilla `interFoam`. The single modification (– `PCV()`) is the canonical diffuse-interface phase-change pressure coupling. **No issues found.** Solver-convergence diagnostics should monitor `p_rghFinal` residuals on fine meshes – if residuals stall, raise `nNonOrthogonalCorrectors` or tighten `tolerance`, not introduce empirical relaxation.

19.3 Joint `UEqn-pEqn` consistency

Item	UEqn	pEqn	Consistent?
rho field	mixture rho	mixture rho	OK
Surface tension	<code>surfaceTensionForce()</code> in RHS reconstruct	<code>surfaceTensionForce()</code> in <code>phig</code>	OK (Rhie-Chow)
Gravity	<code>ghf*snGrad(rho)</code> in RHS reconstruct	<code>ghf*snGrad(rho)</code> in <code>phig</code>	OK
Stefan flux	absent (correct)	absent in <code>phiHbyA</code> (correct)	OK
Phase-change source	absent (correct)	– <code>PCV()</code> in Laplacian RHS	OK (physically distinct roles)
Update order	predicted	corrected after solve	OK

Both equations are clean, mutually consistent, and standard. The publication-grade audit finds no issues in this pair beyond P3’s fine-mesh pressure-solver tightening recommendation.

20. `alphaEqn` source structure – `Su`, `Sp`, `divU` vs `phiStefan` and PCV

20.1 The user’s suspected redundancy

`alphaSuSp.H` constructs three source fields:

```
tmp<volScalarField> tDivU(phaseChangePtr->PCV()); // = PCV
volScalarField::Internal Su (... , 0); // ZERO (despite comment)
volScalarField::Internal Sp (... , 0); // ZERO
volScalarField::Internal divU (... , tDivU().primitiveField()); // = PCV
```

Injected into `alphaEqn.H`:130–131:

```
fvScalarMatrix alpha1Eqn
(
    fvm::ddt(alpha1)
    + fv::gaussConvectionScheme(mesh, phiCN, upwind<scalar>(mesh, phiCN))
    .fvmDiv(phiCN, alpha1)
)
```

```
== Su + fvm::Sp(Sp + divU, alpha1)
);
```

with `phiCN = phi + phiStefan` (Crank-Nicolson blend optional).

The user’s concern: **Su, Sp, divU, phiStefan, and pEqn’s PCV** all carry some form of “phase-change information.” Is something being double-counted?

The answer requires deriving the conservative form rigorously.

20.2 Mathematical derivation

Per-phase mass conservation, constant phase densities:

$$\begin{aligned} d(\alpha \rho_1)/dt + \text{div}(\alpha \rho_1 U) &= -\dot{m} & (\text{mass leaves phase 1}) \\ d((1-\alpha) \rho_2)/dt + \text{div}((1-\alpha) \rho_2 U) &= +\dot{m} \end{aligned}$$

Divide each by its density:

$$\begin{aligned} \text{(A)} \quad d\alpha/dt + \text{div}(\alpha U) &= -\dot{m}/\rho_1 \\ \text{(B)} \quad d(1-\alpha)/dt + \text{div}((1-\alpha) U) &= +\dot{m}/\rho_2 \end{aligned}$$

Add:

$$0 + \text{div}(U) = \dot{m} (1/\rho_2 - 1/\rho_1) = \text{PCV} \quad (\text{mixture continuity})$$

Expand (A) non-conservatively:

$$\begin{aligned} d\alpha/dt + U \cdot \text{grad}(\alpha) + \alpha \text{div}(U) &= -\dot{m}/\rho_1 \\ d\alpha/dt + U \cdot \text{grad}(\alpha) + \alpha \text{PCV} &= -\dot{m}/\rho_1 \end{aligned}$$

Rearrange back to conservative form keeping the dilatation explicit:

$$d\alpha/dt + \text{div}(\alpha U) = -\dot{m}/\rho_1 + \alpha \text{PCV} \quad \dots (*)$$

Equation (*) is the α equation that must be solved. Two source contributions appear:

- **Source A:** $-\dot{m}/\rho_1 = \alpha_1 \text{Gen}()$ (loss of phase 1 to phase change).
- **Source B:** $+\alpha \text{PCV}$ (volumetric dilatation of phase 1 driven by mixture expansion).

These two are **mathematically distinct** and **not redundant**: A is the direct mass-loss source, B is the dilatation correction reflecting that $\text{div}(U) \neq 0$.

20.3 Two equivalent implementations

There are two ways to discretize (*):

Formulation B (source-based, vanilla style):

```
phiCN = phi                                (no Stefan flux)
RHS    = Su + Sp(divU, alpha)
        = alpha1Gen() + alpha * PCV
        = -Q_pc/(rho_1 h_lv) + alpha * PCV
```

Formulation A (kinematic, current code):

```
phiCN = phi + phiStefan                    (Stefan flux added to convection)
RHS    = Sp(divU, alpha) = alpha * PCV
Su      = 0                               (alpha1Gen absorbed into phiStefan)
```

In Formulation A, the source $-\dot{m}/\rho_1$ is moved into the convection operator as the divergence of an auxiliary flux:

$$\text{div}(\alpha \text{phiStefan}) \sim \alpha \text{div}(\text{phiStefan}) + \text{phiStefan} \cdot \text{grad}(\alpha)$$

In the interface band, $\text{div}(\text{phiStefan})_{\text{cell}} \sim \text{mdotRaw}/\text{rho_int} * V_{\text{cell}}$ (by the Sec.18 cell-volume derivation), so the discrete operator contributes approximately $-\text{mdot}/\text{rho_int}$ to the alpha equation – which replaces the $-\text{mdot}/\text{rho_1}$ source (modulo the small rho_int vs rho_1 distinction in the diffuse band).

20.4 Where redundancy WOULD live, and why it doesn't

Redundancy check 1: Su and phiStefan. If Su were `alpha1Gen()` (non-zero) AND `phiCN` included `phiStefan`, the source $-\text{mdot}/\text{rho_1}$ would be applied twice: once explicitly via Su, once kinematically via $\text{div}(\text{alpha } \text{phiStefan})$. The current code zeroes Su (`alphaSuSp.H:11-16`) precisely to avoid this. **Not redundant by design.**

The comment in `alphaSuSp.H:1-3` is stale and misleading:

```
// Phase-change sources for MULES (volScalarField::Internal, [1/s])
// Su = alpha1Gen: positive for condensation, negative for evaporation
```

This describes Formulation B; the code implements Formulation A. The comment should read:

```
// Phase-change sources for MULES.
// Su = 0: alpha1Gen is absorbed into phiStefan in alphaEqn.phiCN.
// Sp = 0: no implicit phase-change linearisation needed.
// divU = PCV: volumetric dilatation source for the alpha*div(U) term.
```

Redundancy check 2: PCV in pEqn vs PCV in alphaEqn. Distinct roles:

- `pEqn-PCV`: enforces $\text{div}(\mathbf{U}) = \text{PCV}$ on the velocity field (mixture continuity).
- `alphaEqn-PCV` (via $\text{div}(\mathbf{U})$): provides the $+\text{alpha} * \text{PCV}$ dilatation term in equation (*).

The first constrains a *velocity field divergence*, the second is a *source coefficient on alpha*. Mathematically independent operators that happen to share the same coefficient $\text{PCV}(\mathbf{x})$ because both ultimately trace to the mass-conservation constraint $\text{div}(\mathbf{U}) = \text{mdot} * (1/\text{rho_v} - 1/\text{rho_1})$. **Not redundant.**

Redundancy check 3: alpha1Gen base-class implementation vs current usage. `thermalPhaseChangeModel::alpha1Gen` is defined in the base class (`thermalPhaseChangeModel.C:117-138`) and returns $-\text{Q_pc}/(\text{rho_1 } h_{lv})$. It is **never called** by the current solver (`alphaSuSp.H` hardcodes $\text{Su}=0$). This is dead infrastructure – it would matter only if Formulation B were used, which the solver does not. See Sec.7.1 for the architectural recommendation: either delete `alpha1Gen` from the base class or document it as unused.

20.5 Quantitative consistency check

Integrating equation (*) over the interface band:

$$\begin{aligned} \frac{d}{dt} \int_{\text{band}} \alpha \, dV \\ = - \int_{\text{band}} \text{div}(\alpha (\mathbf{U} + \mathbf{U}_s)) \, dV \\ + \int_{\text{band}} \alpha \, \text{PCV} \, dV \end{aligned}$$

The first term, by Gauss, equals the net inward flux through the band boundary. In a closed evaporation problem, the band is interior and the fluxes through its outer surface balance the kinematic motion of the interface. The second term equals approximately $\alpha * \text{mdot} * (1/\text{rho_v} - 1/\text{rho_1})$, the volumetric expansion of the liquid ($\alpha=1$ side) into newly created vapor space.

Mass conservation in liquid mass $M_{\text{liq}} = \int \alpha \, \text{rho_1} \, dV$:

$$\begin{aligned} \frac{dM_{\text{liq}}}{dt} &= \text{rho_1} * \int \frac{d\alpha}{dt} \, dV \\ &= \text{rho_1} * [-\text{flux} + \int \alpha \, \text{PCV} \, dV] \\ &= -\text{rho_1}/\text{rho_1} * \int \Gamma \, \text{mdot} \, dA \quad (\text{by Sec.20.3}) \\ &\quad + \text{rho_1} * (\text{mdot} * (1/\text{rho_v} - 1/\text{rho_1}))_{\text{int_band}} \\ &= -\int \text{mdot} \, dA + \text{corrections from dilatation term} \end{aligned}$$

A precise audit requires the actual `phiStefan` divergence integral, but the structural result is: the kinematic `phiStefan` replaces the `-mdot/rho_1` source one-for-one, and the `alpha*PCV` term is the volume-correction completing mass conservation.

20.6 What IS suspect in `alphaSuSp.H` / `alphaEqn`

Not redundancy – but adjacent issues:

S1. `Sp = 0` always, but the MULES API expects `Sp` to be the implicit-source diagonal coefficient. Hardcoding to zero forces MULES to bound `alpha` based solely on explicit `Su + alpha*divU`. This is acceptable but discards a degree of freedom that could improve boundedness during strong phase change. **No action required**, but document the choice.

S2. `divU = PCV` injection via `tmp<volScalarField>` and immediate `.primitiveField()` strip is awkward. A direct construction:

```
volScalarField::Internal divU
(
    IOobject(...),
    mesh, dimless/dimTime,
    phaseChangePtr->PCV()().primitiveField()
);
```

without the named `tmp` is cleaner and equivalent. **Minor cleanup.**

S3. The commented-out term – `fvm::Sp(fvc::ddt(1) + fvc::div(phiCN), alpha1)` (`alphaEqn.H:128-129`). This would discretely add $-(d/dt + \text{div}(\phi_{CN})) \cdot \alpha_1$ to the LHS, converting the conservative `div(alpha1 phiCN)` form into a non-conservative `alpha1 * (d/dt + div(phiCN))` correction. With `divU = PCV` on the RHS this would **double-count the dilatation**, since the commented term embeds `div(phiCN)` which already includes PCV. Correctly left commented. **Delete the comment to avoid confusion.**

S4. `tSu` allocates a `volScalarField` only to extract its `primitiveField()` for `Su`. Wasted allocation. Replace with direct `Su = 0` initialization (already dimensioned correctly):

```
volScalarField::Internal Su
(
    IOobject(...), mesh, dimensionedScalar("Su", dimless/dimTime, Zero)
);
```

20.7 Verdict

Suspected redundancy	Verdict
<code>Su + phiStefan</code> both carrying phase-change source	Not redundant; <code>Su=0</code> by design (Formulation A)
PCV in <code>pEqn</code> AND <code>alphaEqn</code>	Not redundant; mathematically distinct roles
<code>alpha1Gen()</code> base-class accessor	Dead in current solver; remove or document
Comment claims <code>Su=alpha1Gen</code>	Stale comment ; rewrite per Sec.20.4
Commented <code>fvm::Sp(div(phiCN), alpha1)</code>	Correctly inactive; remove comment
<code>tSu</code> temp-field allocation	Minor cleanup possible

The architecture is **mathematically clean**: Formulation A (kinematic) is consistently applied, no double-counting exists, PCV's two roles are physically distinct. The perception of redundancy comes from (i) stale comments still describing Formulation B, (ii) dead `alpha1Gen()` base-class code, and (iii) the cosmetic noise of `tSu/tDivU` temporaries. Cleanup is documentation and dead-code removal, **not equation modification**.

21. Acoeff in TEqn – what it actually does

The Acoeff block in TEqn.H is the most opaque construction in the solver. This section breaks it down from first principles.

21.1 What you see in the code

```
const dimensionedScalar Tsat = phaseChangePtr->T_sat();
const dimensionedScalar eps_T("eps_T", dimTemperature, 2.5);
const scalar implicitFactor = 0.5;

const volScalarField Acoeff
(
    implicitFactor
    * max(
        Q_pc_th_now / max(T - Tsat, eps_T),
        dimensionedScalar("zero", dimensionSet(1,-1,-3,-1,0,0,0), 0)
    )
);

// ... and in the matrix:
+ fvm::Sp(Acoeff, T)
== Acoeff*Tsat
```

(Ignoring the floor terms for now – those are added on top of this.)

This looks like a strange combination of a divide, two maxes, a saturation constant, and an implicit/explicit pair that share the same coefficient. Let me unpack it.

21.2 The trick being used: implicit source linearization

The physical latent-heat source in the energy equation is:

$$\rho C_p \frac{dT}{dt} + \dots - \text{div}(k \text{ grad } T) = -Q_{pc}$$

$Q_{pc} = \dot{m} h_{lv} > 0$ for evaporation/melting (energy absorbed by phase change). If we just slap $-Q_{pc}$ on the RHS as a fully explicit source, the equation is stable only when dt is small enough that T cannot overshoot T_{sat} in a single step. On reasonable engineering meshes with Q_{pc} of order 10^8 W/m^3 in interface cells, this is restrictive.

The standard OpenFOAM trick is to **factor** the source as $A*(T - T_{sat})$:

$$-Q_{pc} = -A * (T - T_{sat}) \quad \text{where} \quad A = Q_{pc} / (T - T_{sat})$$

This identity is true at every point where $T \neq T_{sat}$. The reason for factoring it is that **$-A*(T - T_{sat})$ can be split implicit/explicit**:

$$\begin{array}{rcc} -A*(T - T_{sat}) & = & -A*T + A*T_{sat} \\ & & \text{~~~~~} \quad \text{~~~~~} \\ & & \text{LHS} \quad \text{RHS} \quad (\text{both implicit in } T \text{ via } fvm::Sp) \end{array}$$

In OpenFOAM matrix form:

```
+ fvm::Sp(A, T)    // adds +A*T to LHS (matrix diagonal)
== ... + A*Tsat;   // adds +A*Tsat to RHS
```

After rearranging, the net contribution to the RHS is $A*T_{sat} - A*T = -A*(T - T_{sat}) = -Q_{pc}$. **Mathematically equivalent to `== -Q_pc`**, but with two stability advantages:

1. $+A*T$ enlarges the diagonal of the matrix $\Rightarrow$ Better-conditioned linear system, faster GAMG/PCG convergence.

2. If T overshoots above T_{sat} in one iteration, $+A*T$ immediately pulls it back – implicit damping. Pure-explicit $-Q_{\text{pc}}$ cannot do this; it just blindly subtracts a fixed value.

This is identical in structure to radiation linearization, drag-force linearization, and many other OpenFOAM source patterns. The pattern is:

```
explicit source S(T) --> fvm::Sp(dS/dT, T) == dS/dT * T_0 + S(T_0)
```

with T_0 chosen so that $dS/dT * T_0$ cancels the implicit linearisation. Here $T_0 = T_{\text{sat}}$.

21.3 Three protective max operations

The naive choice $A = Q_{\text{pc}} / (T - T_{\text{sat}})$ breaks at three places:

Issue 1: $T = T_{\text{sat}}$ exactly. Division by zero. Fixed by the inner max:

```
denom = max(T - Tsat, eps_T), eps_T = 2.5 K
```

When T is within 2.5 K above T_{sat} , the denominator is clipped to eps_T . The A_{coeff} becomes $0.5 * Q_{\text{pc}} / 2.5$, finite. Acts as a soft saturation in the singularity neighborhood.

Issue 2: $T < T_{\text{sat}}$. Then $T - T_{\text{sat}} < 0$. Without the max, the denominator would be negative, making $A < 0$. A negative diagonal contribution destabilises the linear solver (loss of diagonal dominance) and physically would *push* T further away from T_{sat} . Fixed by the inner $\max(T - T_{\text{sat}}, \text{eps}_T)$ clipping to eps_T (positive).

Issue 3: $Q_{\text{pc}} < 0$ (condensation, hypothetically). Then $A < 0$ for the same diagonal-dominance reason. Fixed by the outer $\max(\dots, 0)$, which clips negative A to zero – in condensation cells A_{coeff} is just zero, and no implicit term is added. (In the current code Q_{pc} is always ≥ 0 , but the guard is there.)

Net protected formula:

```
A = max(Q_pc / max(T - Tsat, 2.5 K), 0)
```

21.4 The implicitFactor = 0.5 mystery

Now the awkward part. If we apply the full identity $-Q_{\text{pc}} = -A*(T - T_{\text{sat}})$, we should use $A = Q_{\text{pc}}/(T - T_{\text{sat}})$ (i.e. $\text{implicitFactor} = 1$). The code uses 0.5. What does this mean?

Trace the effective RHS contribution:

```
+ fvm::Sp(Acoeff, T) == Acoeff * Tsat
```

is equivalent on the RHS to:

```
Acoeff * Tsat - Acoeff * T = -Acoeff * (T - Tsat)
                        = -0.5 * Q_pc / max(T-Tsat, 2.5) * (T - Tsat)
```

Two regimes:

Regime	$T - T_{\text{sat}}$	$A_{\text{coeff}}*(T - T_{\text{sat}})$ effective sink	Magnitude vs Q_{pc}
Strongly superheated	$T - T_{\text{sat}} > 2.5 \text{ K}$	$0.5 * Q_{\text{pc}}$	Half of Q_{pc}
Near-saturation	$0 < T - T_{\text{sat}} < 2.5 \text{ K}$	$0.5 * Q_{\text{pc}} * (T - T_{\text{sat}}) / 2.5$	$< \text{half}$
Subcooled	$T < T_{\text{sat}}$	$0.5 * Q_{\text{pc}} * (T - T_{\text{sat}}) / 2.5 < 0$	<i>Heat source</i> , pushes T up to T_{sat}

So in the strongly-superheated regime, **the A_{coeff} term applies only HALF of the latent heat as a sink**. The other half of Q_{pc} is **not present** anywhere else in the equation – there is no $-0.5*Q_{\text{pc}}$ on the RHS to make up the deficit.

This is not a bug in the sense of an oversight – it is a deliberate **stabilisation strategy**, but it deserves to be made explicit. The interpretation is:

Acoeff is a “soft Dirichlet” penalty enforcing $T = T_{\text{sat}}$ in the diffuse band, NOT a literal latent-heat sink. The literal latent heat is delivered to the equation by the *conductive flux from the hot side into a cell pinned at T_{sat}* : once T at the interface is constrained to T_{sat} , the Laplacian $-\text{div}(\mathbf{k} \text{ grad } T)$ automatically carries the correct amount of energy across the interface. The Stefan condition $\mathbf{k} \, dT/dn = \dot{m} \cdot h_{\text{lv}}$ then holds at the constrained surface as an emergent property, not as an imposed source.

In this interpretation:

- The role of **Acoeff** is to **constrain T** , not to **balance latent heat magnitude**.
- The magnitude $0.5 \cdot Q_{\text{pc}} / (T - T_{\text{sat}})$ need only be large enough to produce strong enough damping that T converges to T_{sat} in the band within a few outer iterations.
- **implicitFactor = 0.5** is then a **relaxation parameter** controlling how stiff the soft Dirichlet is. Larger $\Rightarrow$ stiffer constraint, faster convergence to $T=T_{\text{sat}}$, but worse conditioning. Smaller $\Rightarrow$ softer constraint, slower pin to T_{sat} , but the bulk solver is happier.

21.5 Why the soft-Dirichlet view is physically defensible

For a sharp-interface Stefan problem with $T = T_{\text{sat}}$ at the interface imposed as a Dirichlet boundary condition:

$$\begin{aligned} -\text{div}(\mathbf{k} \text{ grad } T) &= 0 && \text{in each phase} \\ T &= T_{\text{sat}} && \text{on } \Gamma \\ \mathbf{k}_{\text{hot}} \cdot dT/dn|_{\Gamma} &= \dot{m} \cdot h_{\text{lv}} && (\text{Stefan condition closure}) \end{aligned}$$

The latent heat does not appear as a volumetric source. It is delivered by the *flux through the Dirichlet boundary*: the conductive flux into the constraint surface equals $\dot{m} \cdot h_{\text{lv}}$ automatically when the temperature field, conductivity, and $\dot{m}$ are mutually consistent.

In a diffuse-interface VOF setting we cannot impose a Dirichlet BC on a moving surface, so we approximate it with a volumetric penalty:

$$"T = T_{\text{sat}}" \quad \text{--(approximate)--} \quad + \text{fvm::Sp}(A_{\text{large}}, T) == A_{\text{large}} \cdot T_{\text{sat}}$$

The larger A_{large} , the closer T sticks to T_{sat} in the band. Once T is pinned, the conductive flux delivers latent heat naturally, exactly as in the sharp-interface case. The **Acoeff = $Q_{\text{pc}} / (T - T_{\text{sat}})$** choice **adaptively sets A_{large} based on the local phase-change activity**: where Q_{pc} is strong, A is large, the constraint is tight; where Q_{pc} is weak, A is small, the constraint is loose. This is a smart self-scaling choice and the reason this formulation works in practice.

21.6 The Acoeff_floor and Acoeff_hot_floor add-ons

The two floor terms in TEqn.H complement Acoeff:

Coefficient	Form	Role
Acoeff	$0.5 \cdot Q_{\text{pc}} / \max(T - T_{\text{sat}}, 2.5)$	Active where $Q_{\text{pc}} > 0$; soft Dirichlet at T_{sat}
Acoeff_floor	$5 \cdot \rho \, C_p / dt \cdot 4 \alpha (1 - \alpha) \cdot \text{pos}(T_{\text{sat}} - T)$	Active where $T < T_{\text{sat}}$ in diffuse band; pulls T UP to T_{sat}
Acoeff_hot_floor	$5 \cdot \rho \, C_p / dt \cdot 4 \alpha (1 - \alpha) \cdot \text{pos}(T - T_{\text{sat}}) \cdot \text{coldSideGate}$	Active where $T > T_{\text{sat}}$ in cold-phase side of diffuse band; pulls T DOWN to T_{sat}

Same implicit $\text{Sp}(A, T) == A \cdot T_{\text{sat}}$ structure for each, so each is a soft Dirichlet at T_{sat} . They differ in *when they activate*:

- `Acoeff` activates where there is positive `Q_pc` (phase change is driving heat exchange).
- `Acoeff_floor` activates where `T` has fallen below `Tsat` in the band (anti-trench: prevents sub-saturation cold spots).
- `Acoeff_hot_floor` activates where `T` exceeds `Tsat` on the cold side (anti-superheat: prevents thermal leakage into the cold phase).

The two floors are the **dt-dependent** terms flagged in Sec.13-14 as numerical bandaids. Their $\rho C_p / dt$ scaling makes them infinitely stiff as $dt \rightarrow 0$, which is the publication red flag.

21.7 Summary diagram

TEqn LHS:

```
rho Cp dT/dt + div(rho Cp phi T) - div(k grad T)
+ Sp(Acoeff, T)      <-- soft Dirichlet @ Tsat, where Q_pc > 0
+ Sp(Acoeff_floor, T) <-- soft Dirichlet @ Tsat, anti-trench (cold side)
+ Sp(Acoeff_hot_floor, T) <-- soft Dirichlet @ Tsat, anti-leakage (hot side, cold phase)
```

TEqn RHS:

```
Acoeff * Tsat
+ Acoeff_floor * Tsat
+ Acoeff_hot_floor * Tsat
- Q_cond      <-- condensation explicit branch (zero in evap)
+ Qcorr       <-- enthalpy correction (HW eq. 42)
```

All three implicit penalties enforce the same constraint $T \rightarrow Tsat$ in different cells. The latent heat itself is not subtracted as a literal source – it is delivered emergently by the Laplacian flux into the constrained band.

21.8 What to change for publication

Recommended cleanup of `Acoeff`:

1. **Remove `implicitFactor = 0.5`.** Use $A = Q_{pc} / \max(T - Tsat, \epsilon_{ps_T})$ directly. The 0.5 is a relaxation parameter for which no justification exists in the code; documenting its origin or removing it improves defensibility. If empirical tuning is required for stability, do it via `TEqn.relax()` (which has a dictionary-set under-relaxation factor) rather than hardcoded scaling.
2. **Reduce `eps_T = 2.5 K` to ~ 0.5 K.** The denominator floor is a regularizer for the singularity at $T = Tsat$. 2.5 K is large enough to significantly weaken `Acoeff` in the typical superheat range of evaporation problems. A smaller value gives a tighter Dirichlet at the cost of stiffer matrices – worth doing because the matrix is solved by GAMG which handles stiffness well.
3. **Remove `Acoeff_floor` and `Acoeff_hot_floor`** once the qn reconstruction (Sec.4.3) eliminates the underlying noise that currently makes them necessary. Their $\rho C_p / dt$ scaling is the only dt-dependent regularisation in the solver and is publication-blocking (Sec.14).
4. **Rename the variable to make intent explicit.** `Acoeff` reads as “some coefficient A”; `softDirichletAtTsat` documents the physical role.

Pedagogical comment block to add inline:

```
// Implicit "soft Dirichlet" penalty enforcing T -> Tsat in the diffuse
// interface band. Mathematical structure: fvm::Sp(A, T) == A * Tsat is
// equivalent to a RHS source of -A*(T-Tsat). Choosing A = Q_pc/(T-Tsat)
// makes the penalty strength scale with local phase-change activity.
// The latent heat itself is NOT subtracted explicitly here -- it is
// delivered by the Laplacian -div(k grad T) flowing into the Tsat-pinned
// band, which is the diffuse-interface analogue of the sharp-interface
// Stefan-condition closure.
```

21.9 Connection back to the rest of the solver

The Acoeff design assumes:

- `Q_pc_thermal` is the latent activity field driving the penalty.
- The harmonic `kappaf` carries the conductive flux into the `Tsat`-pinned band.
- `phiStefan` carries the kinematic interface motion separately.
- `Qcorr` removes the artificial enthalpy if the Helmholtz redistribution is active.

If any of these is missing or inconsistent, Acoeff cannot achieve its soft-Dirichlet role and the floors become necessary. **The publication-grade fix is to make all four upstream components physically consistent (per Sec.3 and Sec.4 recommendations), after which Acoeff alone – with `implicitFactor = 1.0`, `eps_T ~ 0.5 K`, no floors – is sufficient.**

22. `Q_cond` – is it required for a generalized bidirectional solver?

Short answer: no. `Q_cond` exists only because the current Acoeff construction has a `max(..., 0)` outer guard that zeroes the implicit penalty whenever `Q_pc < 0`. With a sign-aware denominator, a single Acoeff handles both evaporation/melting (`Q_pc > 0`) and condensation/freezing (`Q_pc < 0`) uniformly, and `Q_cond` becomes unnecessary.

22.1 Why the current split exists

```
Acoeff = max(Q_pc / max(T - Tsat, eps_T), 0);    // zero when Q_pc < 0
Q_cond = min(Q_pc, 0);                          // explicit branch for negatives
```

- `Q_pc > 0` (evap/melt): Acoeff > 0, implicit pull-to-Tsat active, latent sink delivered via `+Sp(A, T) == A*Tsat`.
- `Q_pc < 0` (cond/freeze): Acoeff = 0, no implicit term; `-Q_cond` on RHS adds the released latent heat explicitly.

The split is symptomatic, not principled – the sign-handling of Acoeff is simply incomplete.

22.2 Bidirectional Acoeff – one expression for both signs

Build a denominator that carries the sign of `Q_pc` and stays bounded away from zero:

```
const volScalarField sQ = sign(Q_pc);

// denom has the same sign as Q_pc; magnitude is at least eps_T
const volScalarField denom
(
    sQ * max(sQ*(T - Tsat),
             dimensionedScalar("eps_T", dimTemperature, 0.5))
);

// Bidirectional Acoeff: ratio of same-signed quantities -> always >= 0
const volScalarField Acoeff = Q_pc / denom;
```

Regime table:

Q_{pc}	$T - T_{sat}$	sQ	$sQ*(T-T_{sat})$	denom	$A_{coeff} = Q_{pc}/denom$	$Sp(A,T) == A*T_{sat}$ effective source
> 0	≥ 0	+	$T - T_{sat}$	$T - T_{sat}$	$Q_{pc}/(T-T_{sat})$	$-Q_{pc}$ (evap sink)
> 0	< 0 (off-eq)	+	$< 0 \rightarrow$ clipped to 0.5	+0.5	$Q_{pc}/0.5$	> 0 weak penalty pulling T up
< 0	≤ 0	-	$T_{sat} - T$	$T - T_{sat} < 0$	$Q_{pc}/(T-T_{sat})$	$-Q_{pc} > 0$ (cond source)
< 0	> 0 (off-eq)	-	$< 0 \rightarrow$ clipped to -0.5	-0.5	$Q_{pc}/(-0.5)$	> 0 weak penalty pulling T down

In every regime:

- **Acoeff** ≥ 0 (positive diagonal preserved, solver-stable).
- Effective RHS source = $-A_{coeff}*(T - T_{sat}) = -Q_{pc}$ (correct sign for both evaporation and condensation).
- Off-equilibrium cells (Q_{pc} and $(T-T_{sat})$ of opposite sign, indicating transient mismatch) get a weak penalty pulling T back toward T_{sat} with strength Q_{pc} / ϵ_{ps_T} , which is the same regularization role the current ϵ_{ps_T} clip plays in the unidirectional case.

Q_{cond} is no longer needed – the implicit **Acoeff** handles condensation by itself.

22.3 Why this is preferable to the current split

Issue	Current (split)	Bidirectional
Condensation stability	Pure explicit $-Q_{cond}$ – overshoots when stiff	Implicit damping, same as evaporation
Code paths	Two sign-dependent branches	One expression
Compatibility with $Q_{pc}()$ accessor	Requires sign branching downstream	Self-consistent
Dead-code status (Sec.18 D5)	$Q_{cond} == 0$ today because qn-search forces $Q_{pc} \geq 0$	Becomes live once qn is reformulated (Sec.4.3 LS gradient)

The current Q_{cond} is **dead code in the live solver**: the `HardtWondra.C:740` neighbor-search filter `superheat > 1e-6` forces $Q_{pc} \geq 0$. $Q_{cond} = \min(Q_{pc}, 0)$ is therefore identically zero in every run today. It would only become meaningful after the Sec.4.3 LS-gradient reformulation enables bidirectional Q_{pc} . At that point, the bidirectional **Acoeff** above is the clean replacement – Q_{cond} need never be reintroduced.

22.4 Equivalent cleaner form using pos/neg functions

If you prefer to avoid `sign()` (which has discontinuous derivative at zero), the same result follows from a **pos/neg** split factored into the denominator:

```
const volScalarField denom
(
    pos(Q_pc) * max(T - Tsat,
                    dimensionedScalar("eps_T", dimTemperature, 0.5))
    - neg(Q_pc) * max(Tsat - T,
                    dimensionedScalar("eps_T", dimTemperature, 0.5))
)
```

```
);
const volScalarField Acoeff = Q_pc / denom;
```

$\text{pos}(x) = 0.5 \cdot (1 + \text{sign}(x))$ and $\text{neg}(x) = 0.5 \cdot (1 - \text{sign}(x))$ are smooth where supported by OpenFOAM, and zero on the inactive sign. Same result, slightly more verbose.

22.5 What changes in the cleaned TEqn

```
// REMOVE these:
// const volScalarField Q_cond
// (
//     min(Q_pc_now, dimensionedScalar("zero", ..., 0))
// );

// REPLACE Acoeff with bidirectional version (see 22.2)
const volScalarField sQ = sign(phaseChangePtr->Q_pc());
const volScalarField denom
(
    sQ * max(sQ*(T - Tsat), eps_T)
);
const volScalarField Acoeff = phaseChangePtr->Q_pc() / denom;

// Matrix unchanged -- one less term on RHS:
fvScalarMatrix TEqn
(
    rhoCp * fvm::ddt(T)
  + fvm::div(rhoCpPhi, T)
  - fvm::laplacian(kappaf, T)
  + fvm::Sp(Acoeff, T)           // floors removed per Sec.21.8
  ==
    Acoeff * Tsat
  + phaseChangePtr->Qcorr()      // -Q_cond branch removed
);
```

Net change: ~5 lines removed (Q_cond block), ~3 lines added (signed denominator), one less explicit RHS term.

22.6 Combined with the other Sec.21 cleanups

The fully cleaned TEqn for a generalized bidirectional solver:

```
const dimensionedScalar Tsat = phaseChangePtr->T_sat();
const dimensionedScalar eps_T("eps_T", dimTemperature, 0.5); // was 2.5
const volScalarField Qpc = phaseChangePtr->Q_pc();
const volScalarField sQ = sign(Qpc);

// Signed denominator: same sign as Qpc, magnitude >= eps_T
const volScalarField denom(sQ * max(sQ*(T - Tsat), eps_T));

// Bidirectional soft-Dirichlet penalty (was: max(..., 0) split + Q_cond)
const volScalarField softDirichletAtTsat(Qpc / denom); // implicitFactor=1 absorbed

fvScalarMatrix TEqn
(
    rhoCp * fvm::ddt(T)
  + fvm::div(rhoCpPhi, T)
```

```

- fvm::laplacian(phaseChangePtr->kappaf(), T)
+ fvm::Sp(softDirichletAtTsat, T)
==
    softDirichletAtTsat * Tsat
+ phaseChangePtr->Qcorr()
);
TEqn.relax();
TEqn.solve();

```

Net result vs. the current TEqn.H:

Component	Current	Bidirectional clean
Acoeff	$0.5 * \max(Q_{pc}/\max(T-T_{sat}, 2.5), 0)$ (uni-directional, half-magnitude)	$Q_{pc} / (sQ * \max(sQ * (T-T_{sat}), 0.5))$ (bidirectional, full)
Acoeff_floor (anti-trench)	present, $\sim \rho C_p / dt$	removed
Acoeff_hot_floor	present, $\sim \rho C_p / dt$	removed
Q_cond	$\min(Q_{pc}, 0)$ (dead today)	removed
coldSideGate logic	present (TEqn.H + createFields.H bool flag)	removed
Qcorr	present (HW eq. 42)	retained (only meaningful if Helmholtz is kept)

Roughly 60 lines of TEqn.H collapse to 15. Zero dt -dependent coefficients. Single soft-Dirichlet expression handles all four phase-change directions (evap, cond, melt, freeze).

22.7 Verdict

For a generalized phase-change solver: Q_{cond} is not required. It is a workaround for a sign-incomplete Acoeff. A signed-denominator Acoeff reproduces the same physics in one expression, with implicit damping for both signs of phase change, and removes the explicit condensation branch entirely. Combined with the Sec.21.8 cleanups (`implicitFactor = 1`, `eps_T = 0.5 K`, no floors) and the Sec.4.3 LS qn reformulation (signed gradient), the solver becomes fully bidirectional with a single soft-Dirichlet term and no dead code.